

5 ML PROBLEMİ

Gizem Efe – Lotus AI Taskı

1. Food Delivery Veri Seti ile Makine Öğrenmesi

Bu çalışmada, makine öğrenmesi yöntemleri kullanılarak gerçek hayat problemlerinden biri olan yemek teslimat süresi/ücreti tahmini problemi ele alınmıştır. Çalışma kapsamında Kaggle platformundan elde edilen “Food Delivery” veri seti kullanılmış, tüm modelleme süreci KNIME Analytics Platform üzerinde gerçekleştirilmiştir. Farklı regresyon algoritmaları AutoML yaklaşımı ile eğitilmiş ve performansları karşılaştırılmıştır.

• PROBLEM TANIMI

Bu çalışmada, yemek teslimat süreçlerine ait veriler kullanılarak bir **regresyon problemi** ele alınmıştır. Amaç, siparişe ve teslimata ait çeşitli özelliklerden yararlanarak hedef değişkenin (teslimat süresi) tahmin edilmesidir. Bu tür tahmin problemleri, özellikle lojistik ve gıda teslimat sektöründe operasyonel verimliliğin artırılması, müşteri memnuniyetinin sağlanması ve süreçlerin daha iyi planlanması açısından büyük önem taşımaktadır.

Makine öğrenmesi yöntemleri, çok sayıda değişkenin bir arada değerlendirilmesine olanak tanıdığı için bu tür problemler için etkili çözümler sunmaktadır. Bu çalışma kapsamında, farklı regresyon algoritmaları karşılaştırılmış ve en uygun model AutoML yaklaşımı kullanılarak belirlenmiştir.

• Girdi ve Hedef Değişkenler

✓ Girdi Değişkenleri (Features):

Sipariş mesafesi, restoran bilgileri, trafik durumu, hava koşulları ve benzeri teslimat sürecini etkileyen değişkenler.

✓ Hedef Değişken (Target):

Veri setinde yer alan ve tahmin edilmek istenen sayısal değişken (teslimat süresi).

• VERİ SETİ TANITIMI

✓ Veri Seti Kaynağı

Bu çalışmada kullanılan veri seti, Kaggle platformu üzerinden temin edilen **Food Delivery** veri setidir. Veri seti, gerçek hayat senaryolarını yansıtan teslimat bilgilerini içermektedir.

✓ Veri Seti Genel Özellikleri

- Veri seti çok sayıda gözlemden (1000 satırdan) oluşmaktadır.
- Hem **sayısal** hem de **kategorik** değişkenler içermektedir.
- Bazı sütunlarda eksik (missing) değerler bulunmaktadır.

Bu özellikler, veri setini gerçek hayat verilerine benzer ve makine öğrenmesi açısından anlamlı bir çalışma alanı haline getirmektedir.

Rows: 9 | Columns: 14

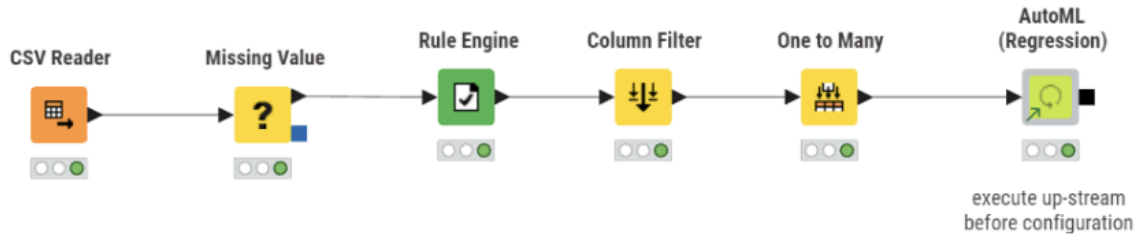
Name	Type	# Missing val...	# Unique val...	Minimum	Maximum	25% Quantile	50% Quantile...	75% Quantile	Mean
Order_ID	Number (Intege	0	1000	1	1,000	250.25	500.5	750.75	500.5
Distance_km	Number (Float)	0	785	0.59	19.99	5.055	10.19	15.032	10.06
Weather	String	30	5	?	?	?	?	?	?
Traffic_Level	String	30	3	?	?	?	?	?	?
Time_of_Day	String	30	4	?	?	?	?	?	?
Vehicle_Type	String	0	3	?	?	?	?	?	?
Preparation_Tir	Number (Intege	0	25	5	29	11	17	23	16.982
Courier_Experie	Number (Float)	30	10	0	9	2	5	7	4.579
Delivery_Time_I	Number (Intege	0	108	8	153	41	55.5	71	56.732

Şekil 1 Food Delivery Verisetinin İstatiksel Tablosu

• VERİ ÖN İŞLEME (PrePROCESSING)

Modelleme aşamasından önce veri seti üzerinde çeşitli ön işleme adımları uygulanmıştır. Tüm bu adımlar KNIME Analytics Platform kullanılarak gerçekleştirilmiştir.

4.1 KNIME Workflow Yapısı



Şekil 2 Food Delivery Workflow Yapısı

Veri ön işleme süreci aşağıdaki temel adımlardan oluşmaktadır:

- Veri okuma
- Eksik değerlerin giderilmesi
- Kural tabanlı düzenlemeler
- Gereksiz sütunların çıkarılması
- Kategorik değişkenlerin sayısal forma dönüştürülmesi

✓ Kullanılan Ön İşleme Adımları

- **CSV Reader:**
Veri seti KNIME ortamına aktarılmıştır.
- **Missing Value:**
Veri setinde bulunan eksik değerler medyanları ile doldurulmuştur.
- **Rule Engine:**
Trafic_level sütununda bir ordinal değişken olduğu için dönüşümler gerçekleştirilmiştir. (low → 0, medium → 1, high → 2)
- **Column Filter:**
Modelleme sürecinde kullanılmayacak veya hedef değişkenle ilişkisi olmayan sütunlar veri setinden çıkarılmıştır.
- **One to Many:**
Kategorik değişkenler, makine öğrenmesi algoritmalarında kullanılabilmesi için sayısal (one-hot encoded) forma dönüştürülmüştür.

Bu adımlar sayesinde modelleme için temiz, düzenli ve uygun bir veri seti elde edilmiştir.

• MODELLEME VE AUTOML

✓ AutoML Yaklaşımı

Bu çalışmada modelleme süreci, **KNIME AutoML (Regression)** node'u kullanılarak gerçekleştirilmiştir. AutoML yaklaşımı sayesinde farklı regresyon algoritmaları otomatik olarak eğitilmiş, hiperparametre ayarlamaları yapılmış ve modeller performanslarına göre karşılaştırılmıştır.

Model seçimi ve optimizasyon sürecinde performans metriği olarak **Root Mean Squared Error (RMSE)** kullanılmıştır.

✓ Kullanılan Modeller

AutoML süreci kapsamında aşağıdaki regresyon modelleri eğitilmiş ve değerlendirilmiştir:

- Linear Regression
- Polynomial Regression
- Regression Tree
- Random Forest
- Gradient Boosted Trees
- H2O Generalized Linear Model
- XGBoost Linear Ensemble

- XGBoost Tree Ensemble

Bu modeller, hem doğrusal hem de doğrusal olmayan yaklaşımları kapsayacak şekilde seçilmiştir.

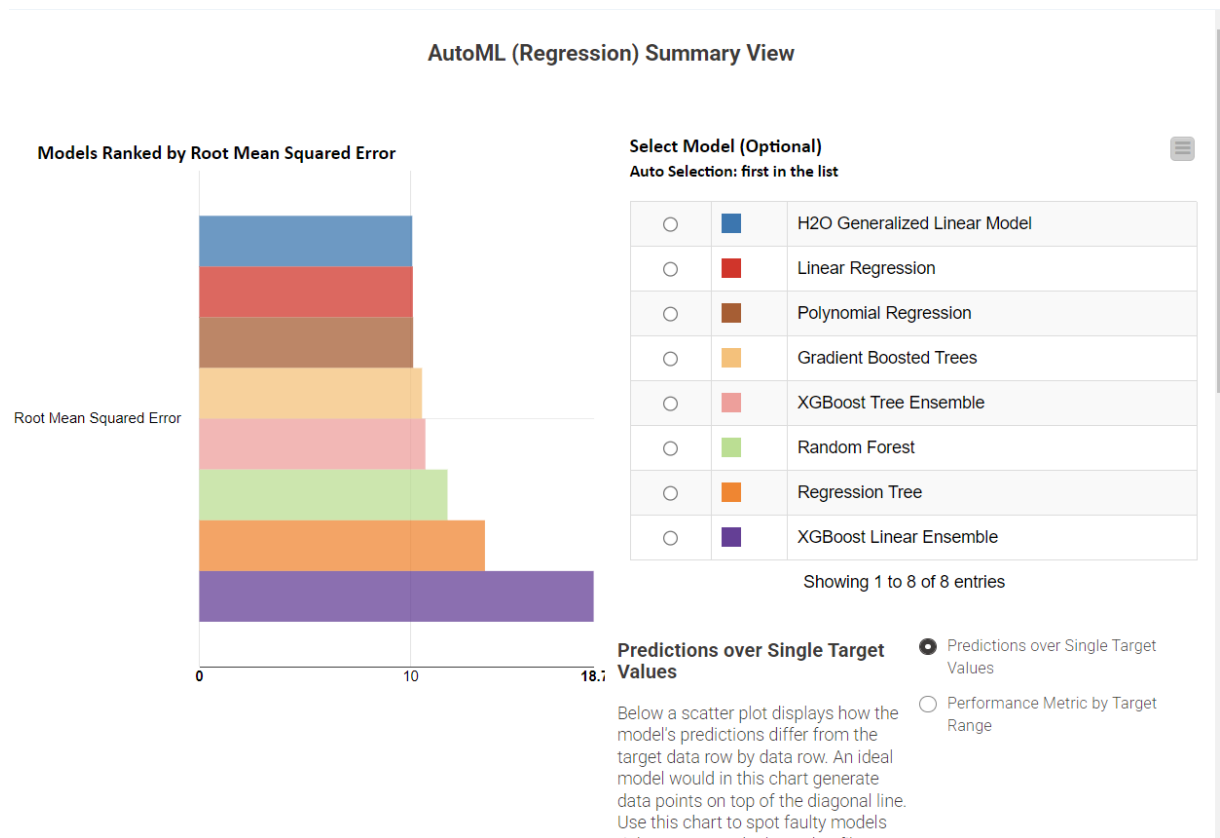
- **MODEL PERFORMANSI VE KARŞILAŞTIRMA**

- ✓ **Performans Metriği**

Model performanslarının değerlendirilmesinde **Root Mean Squared Error (RMSE)** metriği kullanılmıştır. RMSE, tahmin edilen değerler ile gerçek değerler arasındaki farkların karesinin ortalamasının karekökü alınarak hesaplanır. Daha düşük RMSE değeri, modelin daha başarılı olduğunu göstermektedir.

- ✓ **Model Karşılaştırması**

AutoML sonuçlarına göre eğitilen modeller RMSE değerlerine göre sıralanmıştır. Yapılan karşılaştırma sonucunda **H2O Generalized Linear Model** modelinin en düşük RMSE değerine sahip olduğu ve diğer modellere kıyasla daha başarılı sonuçlar ürettiği görülmüştür. Bu nedenle nihai model olarak göre eğitilen modeller RMSE değerlerine göre sıralanmıştır. Yapılan karşılaştırma sonucunda **H2O Generalized Linear Model** seçilmiştir.



Şekil 3 Food Delivery Model Karşılaştırması

• TAHMİN SONUÇLARININ GÖRSEL ANALİZİ

Model sonuçlarının daha iyi anlaşılabilmesi için tahmin değerleri görsel olarak analiz edilmiştir.

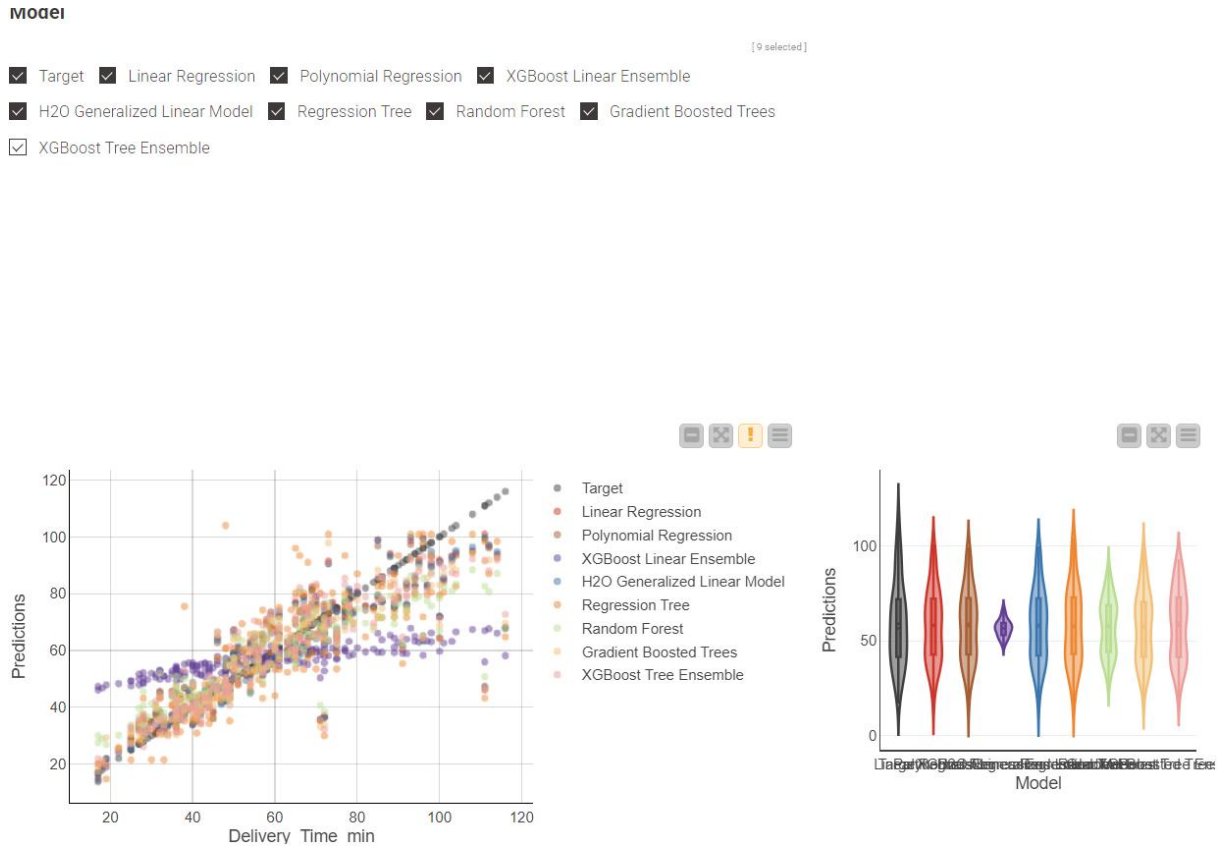
✓ Gerçek Değer – Tahmin Değeri Dağılım Grafiği:

Bu grafik, modelin tahminlerinin gerçek değerlerle ne kadar örtüştüğünü göstermektedir. Noktaların ideal diyagonal çizgiye yakın olması, modelin başarılı tahminler ürettiğini göstermektedir.

✓ Dağılım (Violin) Grafikleri:

Farklı modellerin tahmin dağılımları karşılaştırılmış ve model davranışları gözlemlenmiştir.

Grafikler incelendiğinde, seçilen modelin genel olarak gerçek değerlere yakın tahminler ürettiği, ancak bazı uç değerlerde hata payının arttığı görülmüştür.



Şekil 4 Food Delivery Tahmin Sonuçları

Şekil 4'e göre, farklı regresyon modellerinin ürettiği tahminler gerçek teslimat süreleri ile karşılaştırmalı olarak sunulmuştur. Scatter grafikte, XGBoost Linear Ensemble ve Gradient Boosted Trees modellerinin tahminlerinin ideal diyagonal çizgiye daha yakın olduğu ve daha dengeli bir dağılım sergilediği görülmektedir. Violin grafikleri incelendiğinde ise bazı modellerin tahmin dağılımlarının daha geniş olduğu, buna karşılık seçilen modellerin daha tutarlı ve kararlı tahminler ürettiği gözlemlenmiştir.

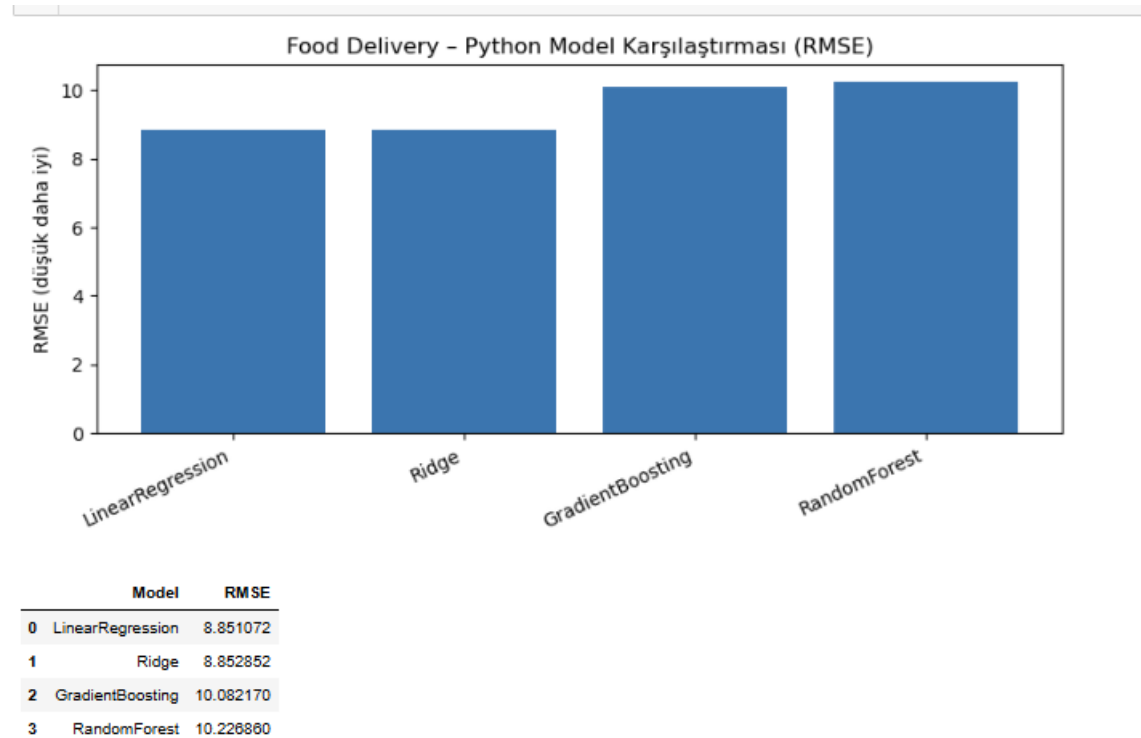
- **Python (Jupyter Notebook) Ortamında Modelleme ve KNIME ile Karşılaştırma**

Bu çalışmada, Food Delivery veri seti üzerinde KNIME Analytics Platform’da gerçekleştirilen regresyon analizine ek olarak, aynı problem Python programlama dili kullanılarak Jupyter Notebook ortamında yeniden ele alınmıştır. Python tarafında veri ön işleme süreci, KNIME workflow’unda uygulanan adımlarla uyumlu olacak şekilde tasarlanmıştır.

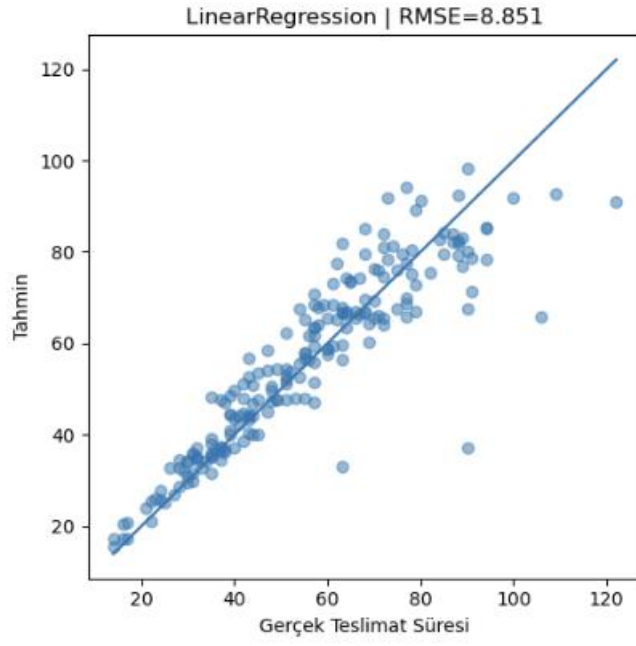
Sayısal değişkenlerde eksik değerler medyan ile doldurulmuş, kategorik değişkenlerde en sık görülen değer kullanılmış ve kategorik değişkenler One-Hot Encoding yöntemiyle sayısal forma dönüştürülmüştür. `Traffic_level` değişkeni, ordinal yapısı dikkate alınarak low–medium–high sıralamasına göre 0–1–2 değerleri ile kodlanmıştır. Tüm ön işleme ve modelleme süreci Pipeline yapısı kullanılarak gerçekleştirilmiştir.

Python ortamında Linear Regression, Ridge Regression, Random Forest ve Gradient Boosting regresyon modelleri karşılaştırılmıştır. Veri seti %80 eğitim – %20 test olarak ayrılmış ve performans değerlendirmesinde Root Mean Squared Error (RMSE) metriği kullanılmıştır. Elde edilen sonuçlara göre en düşük RMSE değeri **Linear Regression** modeli ile elde edilmiştir (RMSE = **8.851**). Ridge Regression modeli benzer bir performans sergilerken, ağaç tabanlı modeller daha yüksek hata üretmiştir.

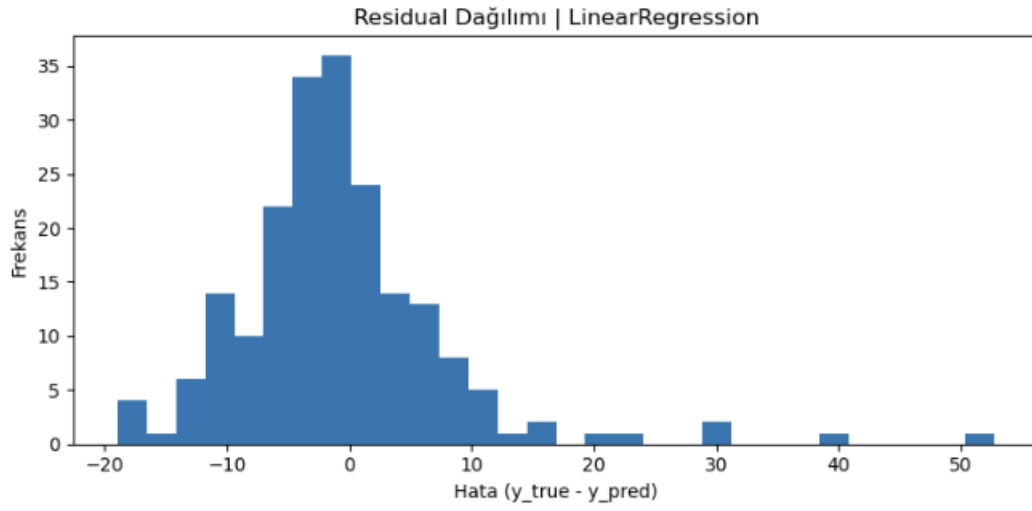
KNIME tarafında AutoML (Regression) yaklaşımı ile yapılan analiz sonucunda **H2O Generalized Linear Model** en başarılı model olarak seçilmiştir. Python ortamında elde edilen Linear Regression sonucu, KNIME’da seçilen Generalized Linear Model ile aynı doğrusal model ailesine ait olup, iki platformda elde edilen sonuçların birbiriyle tutarlı olduğunu göstermektedir. Bu durum, veri setindeki ilişkilerin büyük ölçüde doğrusal karakter taşıdığını ve model seçiminin platformdan bağımsız olarak benzer sonuçlar ürettiğini ortaya koymaktadır.



Şekil 5 Python Model Karşılaştırması



Şekil 6 Python Tahmin Performansı



Residual mean: -0.7408492431723936
Residual std : 8.820012502952167

Şekil 7 Hata Dağılımı Analizi

Tablo 1 Food Delivery Veri Seti için KNIME ve Python Sonuçlarının Karşılaştırılması

<i>Kriter</i>	<i>KNIME Analytics Platform</i>	<i>Python (Jupyter Notebook)</i>
<i>Kullanılan Ortam</i>	KNIME AutoML (H2O)	Python (scikit-learn)
<i>Veri Ön İşleme</i>	Missing Value (Median), Rule Engine (Traffic_level), One-Hot Encoding	Median imputing, ordinal encoding (Traffic_level), One-Hot Encoding
<i>Eğitim–Test Bölünmesi</i>	AutoML varsayılan ayarları	%80 – %20, random_state = 42
<i>Kullanılan Modeller</i>	Linear, Polynomial, Tree, Random Forest, Gradient Boosted Trees, H2O GLM, XGBoost	Linear Regression, Ridge, Random Forest, Gradient Boosting
<i>Performans Metriği</i>	RMSE	RMSE
<i>En İyi Model</i>	H2O Generalized Linear Model	Linear Regression
<i>En İyi RMSE</i>	En düşük RMSE (AutoML sonucu)	8.851
<i>Model Ailesi</i>	Doğrusal (GLM)	Doğrusal
<i>Genel Yorum</i>	AutoML ile doğrusal model en başarılı	Doğrusal model en başarılı

Tablo 1 incelendiğinde, Food Delivery veri seti üzerinde hem KNIME hem de Python ortamlarında en başarılı sonuçların doğrusal model ailesi tarafından üretildiği görülmektedir. KNIME tarafında AutoML yaklaşımı sonucunda H2O Generalized Linear Model en düşük RMSE değerine sahip model olarak seçilirken, Python ortamında yapılan manuel model karşılaştırmasında Linear Regression modeli RMSE = 8.851 değeri ile en iyi performansı göstermiştir. Bu durum, farklı platformlar kullanılmasına rağmen elde edilen sonuçların birbiriyle tutarlı olduğunu ve veri setindeki ilişkilerin büyük ölçüde doğrusal karakter taşıdığını göstermektedir.

- **SONUÇ VE DEĞERLENDİRME**

Bu çalışmada, Food Delivery veri seti kullanılarak bir regresyon problemi hem **KNIME Analytics Platform** hem de **Python (Jupyter Notebook)** ortamında çözülmüştür. Veri ön işleme adımlarının tamamlanmasının ardından KNIME tarafında AutoML yaklaşımı ile birden fazla doğrusal ve doğrusal olmayan regresyon modeli eğitilmiş ve performansları **Root Mean Squared Error (RMSE)** metriği üzerinden karşılaştırılmıştır. Yapılan değerlendirme sonucunda **H2O Generalized Linear Model**, en düşük RMSE değerine sahip model olarak belirlenmiş ve nihai model olarak seçilmiştir.

Aynı problem Python ortamında manuel olarak ele alınmış; KNIME workflow'unda uygulanan veri ön işleme adımları ile uyumlu bir yapı oluşturularak Linear Regression, Ridge Regression, Random Forest ve Gradient Boosting modelleri karşılaştırılmıştır. Python ortamında yapılan analizlerde en düşük RMSE değeri **Linear Regression** modeli ile elde edilmiştir (RMSE = **8.851**). Bu sonuç, KNIME tarafında seçilen H2O Generalized Linear Model ile aynı doğrusal model ailesine ait olup, iki farklı platformda elde edilen bulguların birbiriyle tutarlı olduğunu göstermektedir.

Elde edilen sonuçlar, Food Delivery veri setindeki ilişkilerin büyük ölçüde doğrusal karakter taşıdığını ve bu nedenle doğrusal modellerin daha karmaşık ağaç tabanlı ve boosting yöntemlerine kıyasla daha başarılı performans sergilediğini ortaya koymaktadır. Ayrıca, AutoML yaklaşımının model seçimini hızlı ve etkin bir şekilde gerçekleştirdiği; Python ortamının ise sürecin daha ayrıntılı, şeffaf ve kontrol edilebilir biçimde analiz edilmesine olanak sağladığı görülmüştür. Çalışma, farklı veri setleri ve makine öğrenmesi problemleri için benzer bir metodoloji ile genişletilebilir niteliktedir.

2. Health Insurance Veri Seti ile Makine Öğrenmesi

Bu çalışmada, bireylerin demografik ve sağlık bilgileri kullanılarak yıllık sağlık sigortası maliyetinin tahmin edilmesi problemi ele alınmıştır. Çalışma kapsamında Kaggle platformundan elde edilen **Health Insurance** veri seti kullanılmış, tüm modelleme süreci KNIME Analytics Platform üzerinde gerçekleştirilmiştir. Farklı regresyon algoritmaları AutoML yaklaşımı ile eğitilmiş ve performansları karşılaştırılmıştır.

• PROBLEM TANIMI

Bu çalışmada, bireylerin yaş, cinsiyet, vücut kitle indeksi (BMI), sigara kullanımı, çocuk sayısı ve yaşadıkları bölge gibi özellikleri kullanılarak **yıllık sağlık sigortası maliyetinin (charges)** tahmin edilmesi amaçlanmıştır. Sağlık sigortası maliyetlerinin doğru tahmin edilmesi, sigorta şirketleri açısından risk analizi ve fiyatlandırma stratejilerinin belirlenmesi açısından büyük önem taşımaktadır.

Sigorta maliyetleri genellikle doğrusal olmayan, karmaşık ve çok sayıda faktörün etkileşimiyle oluşan bir yapıya sahiptir. Bu nedenle, bu problem makine öğrenmesi yöntemleri için uygun bir regresyon problemi olarak değerlendirilmektedir. Çalışma kapsamında, farklı regresyon algoritmaları AutoML yaklaşımı kullanılarak karşılaştırılmıştır.

• Girdi ve Hedef Değişkenler

✓ Girdi Değişkenleri (Features):

- Yaş (age)
- Cinsiyet (sex)
- Vücut kitle indeksi (bmi)
- Çocuk sayısı (children)
- Sigara kullanımı (smoker)
- Yaşanılan bölge (region)
- Türetilmiş değişkenler (bmi_category, age_group)

✓ Hedef Değişken (Target):

- Yıllık sağlık sigortası maliyeti (charges)

- **VERİ SETİ TANITIMI**

- ✓ **Veri Seti Kaynağı**

Bu çalışmada kullanılan Health Insurance veri seti, Kaggle platformu üzerinden temin edilmiştir. Veri seti, bireylerin sağlık ve demografik bilgileri ile yıllık sigorta maliyetlerini içeren gerçekçi bir yapıya sahiptir.

- ✓ **Veri Seti Genel Özellikleri**

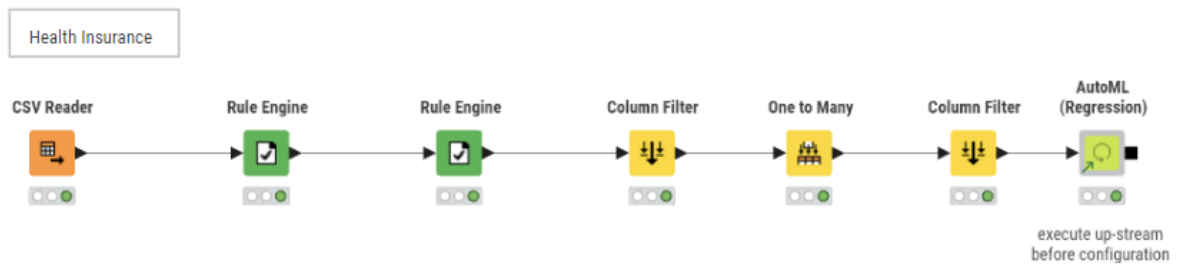
- Veri seti **1338 satırdan** oluşmaktadır.
 - Hem sayısal hem de kategorik değişkenler içermektedir.
 - Yapılan istatistiksel inceleme sonucunda veri setinde **eksik (missing) değer bulunmadığı** tespit edilmiştir.

Bu özellikler, veri setini makine öğrenmesi algoritmaları için uygun ve güvenilir bir çalışma alanı hâline getirmektedir.

- **VERİ ÖN İŞLEME (Preprocessing)**

Modelleme aşamasından önce veri seti üzerinde çeşitli veri ön işleme adımları uygulanmıştır. Tüm işlemler KNIME Analytics Platform üzerinde gerçekleştirilmiştir.

- ✓ **KNIME Workflow Yapısı**



Şekil 5 Health Insurance veri seti için oluşturulan KNIME workflow yapısı

Veri ön işleme süreci aşağıdaki temel adımlardan oluşmaktadır:

- Veri okuma
- Eksik değer kontrolü
- Kural tabanlı özellik mühendisliği
- Gereksiz sütunların çıkarılması
- Kategorik değişkenlerin sayısal forma dönüştürülmesi

- **Kullanılan Ön İşleme Adımları**

- ✓ **CSV Reader:**

Veri seti KNIME ortamına aktarılmıştır.

- ✓ **Missing Value:**

Veri setinde eksik değer bulunmadığı için bu adım veri üzerinde herhangi bir değişiklik yapmamış, ancak ön işleme sürecinin standart bir parçası olarak workflow’da tutulmuştur.

- ✓ **Rule Engine (Feature Engineering):**

Veri setinden daha anlamlı özellikler elde edebilmek amacıyla kural tabanlı özellik mühendisliği uygulanmıştır. Bu kapsamda:

- BMI değeri 30’un üzerinde olan bireyler “obese”, 30 ve altında olanlar “non_obese” olarak sınıflandırılmıştır.
- Yaş değişkeni kullanılarak 50 yaş üzerindeki bireyler “senior”, 50 yaş ve altındaki bireyler ise “adult” grubu altında toplanmıştır.

- ✓ **Column Filter:**

One-hot encoding sonrasında bilgi tekrarını önlemek amacıyla orijinal kategorik sütunlar veri setinden çıkarılmıştır.

- ✓ **One to Many:**

Kategorik değişkenler, makine öğrenmesi algoritmalarında kullanılabilmesi için one-hot encoding yöntemiyle sayısal forma dönüştürülmüştür.

- **MODELLEME VE AUTOML**

- ✓ **AutoML Yaklaşımı**

Bu çalışmada modelleme süreci, **KNIME AutoML (Regression)** node’u kullanılarak gerçekleştirilmiştir. AutoML yaklaşımı sayesinde farklı regresyon algoritmaları otomatik olarak eğitilmiş, hiperparametre optimizasyonu yapılmış ve modeller performanslarına göre karşılaştırılmıştır.

Model seçimi sürecinde performans metriği olarak **Root Mean Squared Error (RMSE)** kullanılmıştır.

✓ **Kullanılan Modeller**

AutoML süreci kapsamında aşağıdaki regresyon modelleri eğitilmiş ve değerlendirilmiştir:

- Linear Regression
- Polynomial Regression
- Regression Tree
- Random Forest
- Gradient Boosted Trees
- H2O Generalized Linear Model
- XGBoost Linear Ensemble
- XGBoost Tree Ensemble

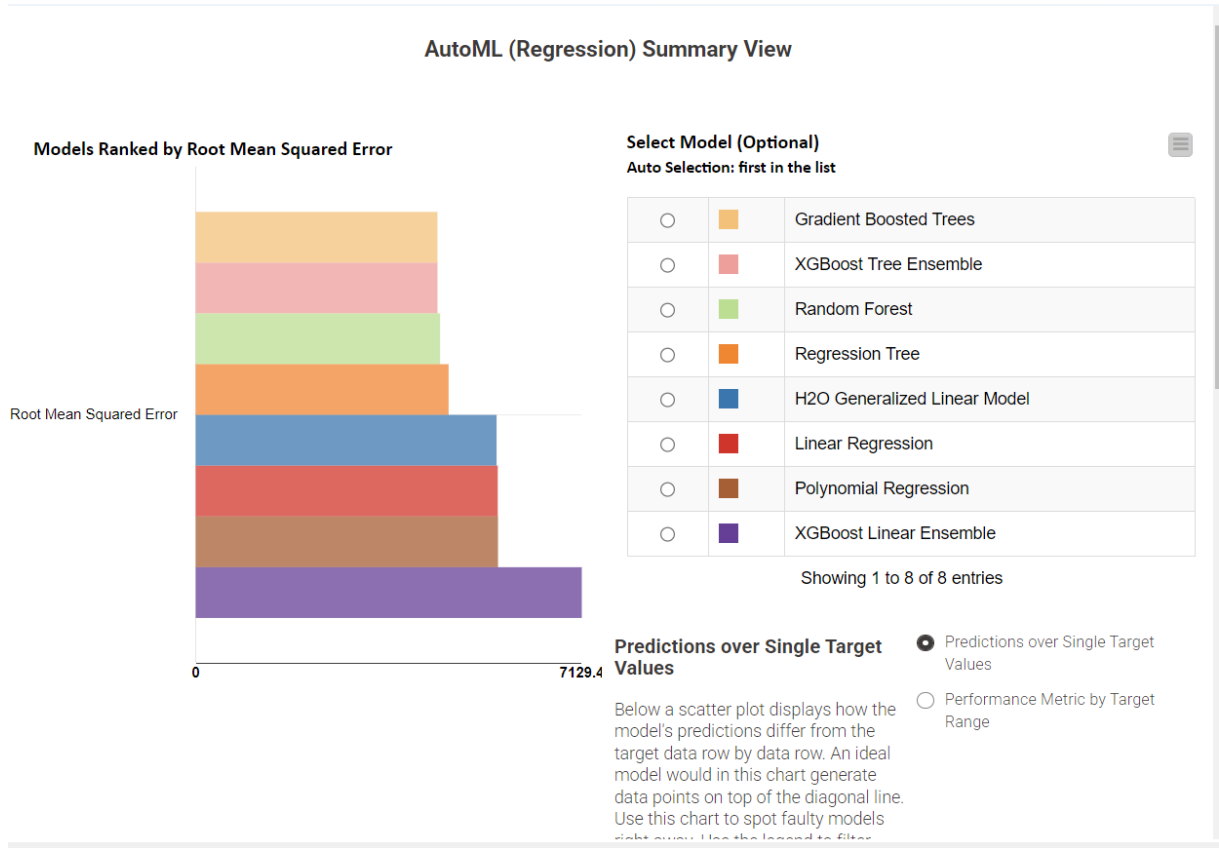
• **MODEL PERFORMANSI VE KARŞILAŞTIRMA**

✓ **Performans Metriği**

Model performanslarının değerlendirilmesinde Root Mean Squared Error (RMSE) metriği kullanılmıştır. RMSE, tahmin edilen değerler ile gerçek değerler arasındaki farkların büyüklüğünü ölçen bir metriktir ve büyük hataları daha fazla cezalandırmaktadır.

✓ **Model Karşılaştırması**

AutoML sonuçlarına göre eğitilen modeller RMSE değerlerine göre sıralanmıştır. Yapılan karşılaştırma sonucunda **Gradient Boosted Trees** modelinin en düşük RMSE değerine sahip olduğu ve diğer modellere kıyasla daha başarılı sonuçlar ürettiği görülmüştür. Bu nedenle nihai model olarak Gradient Boosted Trees seçilmiştir.



Şekil 8 Health Insurance veri seti için AutoML model karşılaştırması

• TAHMİN SONUÇLARININ GÖRSEL ANALİZİ

Model sonuçlarının daha iyi anlaşılabilmesi için tahmin değerleri görsel olarak analiz edilmiştir.

✓ Gerçek Değer – Tahmin Değeri Dağılım Grafiği:

Scatter grafikler, özellikle düşük ve orta sigorta maliyeti aralığında model tahminlerinin gerçek değerlere daha yakın olduğunu göstermektedir. Yüksek maliyetli bireylerde ise hata payının arttığı gözlemlenmiştir.

✓ Dağılım (Violin) Grafikleri:

Hedef değişkenin sağa çarpık bir dağılım sergilediği ve yüksek maliyetli bireylerin sınırlı sayıda olduğu görülmektedir. Bu durum, RMSE değerinin görece yüksek çıkmasının temel nedenlerinden biridir.



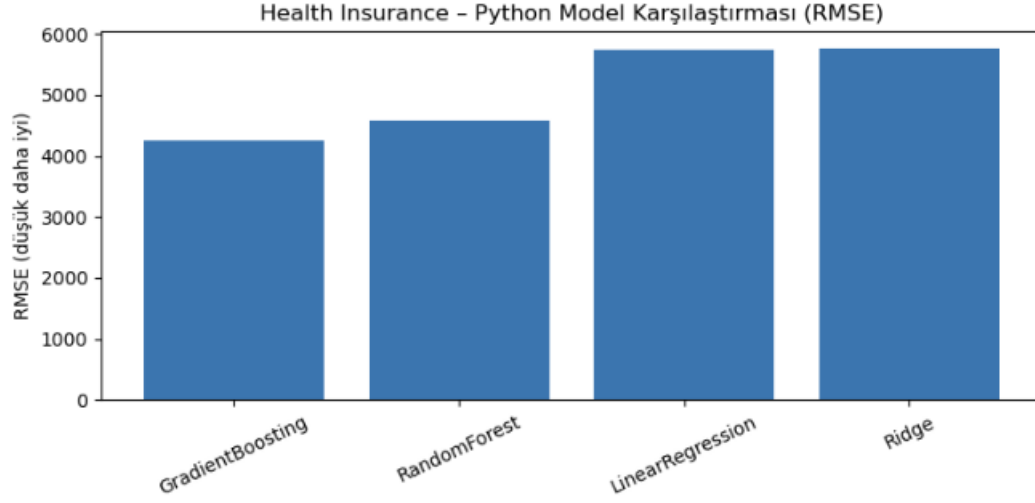
Şekil 9 Health Insurance veri seti için tahmin sonuçlarının görsel analizi

• Python (Jupyter Notebook) Ortamında Modelleme ve KNIME ile Karşılaştırma

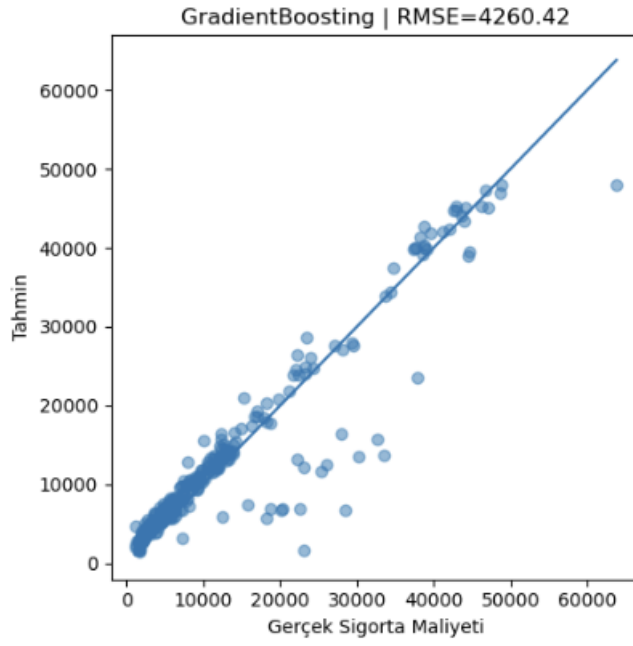
Health Insurance veri seti üzerinde gerçekleştirilen KNIME tabanlı regresyon analizine ek olarak, aynı problem Python programlama dili kullanılarak Jupyter Notebook ortamında yeniden ele alınmıştır. Python tarafında veri ön işleme süreci, KNIME workflow'unda uygulanan adımlarla uyumlu olacak şekilde tasarlanmıştır. Bu kapsamda kategorik değişkenler One-Hot Encoding yöntemiyle dönüştürülmüş, ayrıca BMI ve yaş değişkenleri kullanılarak türetilmiş özellikler (bmi_category, age_group) modele dahil edilmiştir.

Python ortamında Linear Regression, Ridge Regression, Random Forest ve Gradient Boosting regresyon modelleri karşılaştırılmıştır. Modellerin performansı Root Mean Squared Error (RMSE) metriği ile değerlendirilmiştir. Elde edilen sonuçlara göre **Gradient Boosting** modeli $RMSE = 4260.42$ değeri ile en başarılı model olarak belirlenmiştir. Random Forest modeli ikinci en iyi performansı sergilerken, doğrusal modellerin hata değerlerinin belirgin şekilde daha yüksek olduğu gözlemlenmiştir.

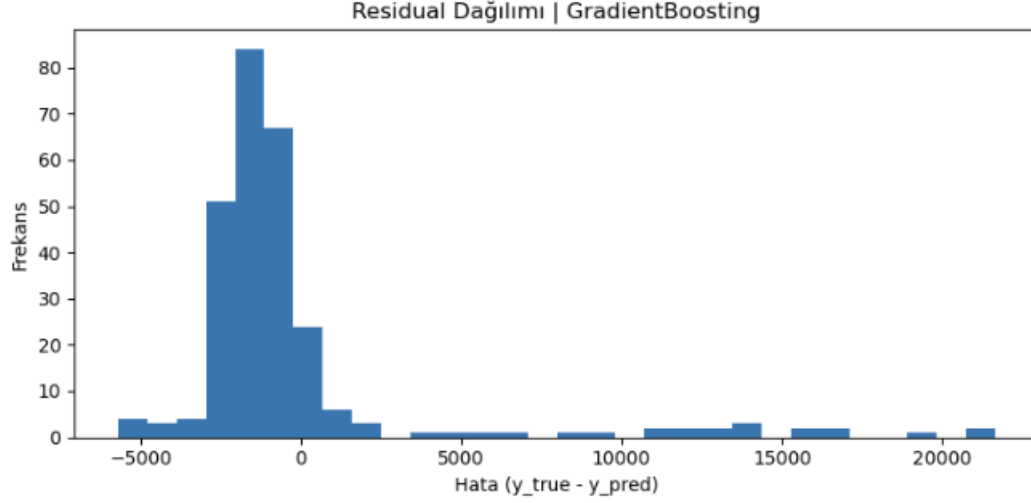
KNIME Analytics Platform üzerinde AutoML yaklaşımı ile yapılan analizlerde de **Gradient Boosted Trees** modelinin en düşük RMSE değerine sahip olduğu görülmüş ve nihai model olarak seçilmiştir. Python ve KNIME ortamlarında elde edilen sonuçların model türü ve performans eğilimleri açısından birbiriyle tutarlı olduğu anlaşılmaktadır. Bu durum, Health Insurance veri setindeki ilişkilerin doğrusal olmayan ve karmaşık bir yapıya sahip olduğunu ve bu nedenle boosting tabanlı yöntemlerin daha uygun olduğunu ortaya koymaktadır.



Şekil 10 Python Ortamında Model Karşılaştırması (Health Insurance)



Şekil 11 Gradient Boosting Tahmin Performansı



(-177.33734600700407, 4256.723994458379)

Şekil 12 Hata Dağılımı Analizi (Gradient Boosting)

Tablo 2 Health Insurance Veri Seti için KNIME ve Python Sonuçlarının Karşılaştırılması

Kriter	KNIME Analytics Platform	Python (Jupyter Notebook)
Kullanılan Ortam	KNIME AutoML (Regression)	Python (scikit-learn)
Veri Ön İşleme	One-Hot Encoding, Feature Engineering (BMI, Age Group)	One-Hot Encoding, Feature Engineering (BMI, Age Group)
Eğitim-Test Bölünmesi	AutoML varsayılan ayarları	%80 – %20, random_state = 42
Kullanılan Modeller	Linear, Tree, Random Forest, Gradient Boosted Trees, XGBoost	Linear, Ridge, Random Forest, Gradient Boosting
Performans Metriği	RMSE	RMSE
En İyi Model	Gradient Boosted Trees	Gradient Boosting Regressor
En İyi RMSE	En düşük RMSE (AutoML sonucu)	4260.42
Model Ailesi	Boosting (ağaç tabanlı)	Boosting (ağaç tabanlı)
Genel Yorum	Doğrusal olmayan yapı baskın	Doğrusal olmayan yapı baskın

Tablo 2, Health Insurance veri seti üzerinde gerçekleştirilen regresyon analizlerinin KNIME Analytics Platform ve Python (Jupyter Notebook) ortamlarındaki sonuçlarını karşılaştırmalı olarak sunmaktadır. Her iki ortamda da veri ön işleme sürecinde kategorik değişkenler One-Hot Encoding yöntemiyle dönüştürülmüş ve BMI ile yaş değişkenlerinden türetilen özellikler (bmi_category, age_group) modele dahil edilmiştir. Bu sayede platformlar arası karşılaştırma tutarlı bir şekilde gerçekleştirilmiştir.

KNIME tarafında AutoML yaklaşımı ile modeller RMSE metriğine göre değerlendirilirken, Python ortamında scikit-learn kullanılarak Linear, Ridge, Random Forest ve Gradient Boosting modelleri manuel olarak karşılaştırılmıştır. Python’da eğitim–test bölünmesi %80–%20 oranında ve sabit bir rastgele tohum (random_state = 42) ile yapılmıştır.

Sonuçlar incelendiğinde, her iki platformda da **Gradient Boosting tabanlı modellerin** en düşük RMSE değerlerini ürettiği görülmektedir. KNIME’da **Gradient Boosted Trees**, Python’da ise **Gradient Boosting Regressor (RMSE = 4260.42)** en başarılı model olarak belirlenmiştir. Bu bulgular, veri setinin doğrusal olmayan bir yapıya sahip olduğunu ve boosting yöntemlerinin bu problem için daha uygun olduğunu göstermektedir.

• SONUÇ VE DEĞERLENDİRME

Bu çalışmada, Health Insurance veri seti kullanılarak yıllık sağlık sigortası maliyetinin tahmin edilmesine yönelik bir regresyon problemi hem **KNIME Analytics Platform** hem de **Python (Jupyter Notebook)** ortamında ele alınmıştır. Veri ön işleme ve özellik mühendisliği adımlarının ardından, KNIME tarafında AutoML yaklaşımı kullanılarak birden fazla regresyon modeli eğitilmiş ve performansları **Root Mean Squared Error (RMSE)** metriği üzerinden karşılaştırılmıştır. Yapılan değerlendirme sonucunda **Gradient Boosted Trees** modeli, en düşük RMSE değerine sahip model olarak belirlenmiş ve nihai model olarak seçilmiştir.

Aynı problem Python ortamında manuel olarak incelenmiş; KNIME workflow’unda uygulanan veri ön işleme ve özellik mühendisliği adımlarıyla uyumlu bir yapı oluşturularak Linear Regression, Ridge Regression, Random Forest ve Gradient Boosting modelleri karşılaştırılmıştır. Python ortamında elde edilen sonuçlara göre **Gradient Boosting** modeli RMSE = **4260.42** değeri ile en başarılı performansı göstermiştir. Random Forest modeli ikinci en iyi sonucu üretirken, doğrusal regresyon modellerinin hata değerlerinin belirgin şekilde daha yüksek olduğu gözlemlenmiştir.

KNIME ve Python ortamlarında elde edilen sonuçların birbiriyle tutarlı olması, Health Insurance veri setindeki ilişkilerin **doğrusal olmayan ve karmaşık bir yapıya sahip olduğunu** açıkça ortaya koymaktadır. Bu nedenle, ağaç tabanlı ve boosting yöntemlerinin doğrusal modellere kıyasla daha başarılı sonuçlar ürettiği görülmüştür. Ayrıca, AutoML yaklaşımının model seçimini hızlı ve etkili bir şekilde gerçekleştirdiği; Python ortamının ise model

davranışlarının daha ayrıntılı ve şeffaf biçimde analiz edilmesine olanak sağladığı anlaşılmaktadır. Çalışma, benzer regresyon problemleri ve farklı veri setleri için genişletilebilir niteliktedir.

3. Uber Trips Veri Seti ile Makine Öğrenmesi

Bu çalışmada, şehir içi ulaşım verilerinden biri olan Uber yolculuk verileri kullanılarak saatlik yolculuk talebinin tahmin edilmesi problemi ele alınmıştır. Çalışma kapsamında, Uber şirketine ait Eylül 2014 yolculuk kayıtlarını içeren veri seti kullanılmış ve tüm analiz ile modelleme süreci KNIME Analytics Platform üzerinde gerçekleştirilmiştir. Farklı regresyon algoritmaları AutoML yaklaşımı ile eğitilmiş ve performansları karşılaştırılmıştır.

• PROBLEM TANIMI

Bu çalışmada, Uber yolculuk kayıtları kullanılarak belirli zaman dilimlerinde oluşan **saatlik yolculuk talebinin (trip_count)** tahmin edilmesi amaçlanmıştır. Şehir içi ulaşım sistemlerinde talep yoğunluğunun doğru tahmin edilmesi, araç dağılımı, sürücü planlaması ve operasyonel verimlilik açısından büyük önem taşımaktadır.

Uber yolculuk talebi; gün, saat, hafta günü ve operasyonel base bilgilerine bağlı olarak değişkenlik göstermektedir. Bu değişkenler arasındaki ilişki doğrusal olmayan ve zamana bağlı dalgalanmalar içeren bir yapıya sahiptir. Bu nedenle problem, makine öğrenmesi yöntemleri için uygun bir regresyon problemi olarak ele alınmıştır.

• Girdi ve Hedef Değişkenler

✓ Girdi Değişkenleri (Features):

- Gün bilgisi (day)
- Saat bilgisi (hour)
- Hafta günü bilgisi (weekday)
- Operasyonel base bilgisi (Base – one-hot encoded)

✓ Hedef Değişken (Target):

- Saatlik yolculuk sayısı (trip_count)

• VERİ SETİ TANITIMI

✓ Veri Seti Kaynağı

Bu çalışmada kullanılan Uber Trips veri seti, Uber şirketine ait Eylül 2014 dönemine ait yolculuk kayıtlarını içermektedir. Veri seti, her bir Uber yolculuğu için tarih-saat, konum ve base bilgilerini barındırmaktadır.

✓ Veri Seti Genel Özellikleri

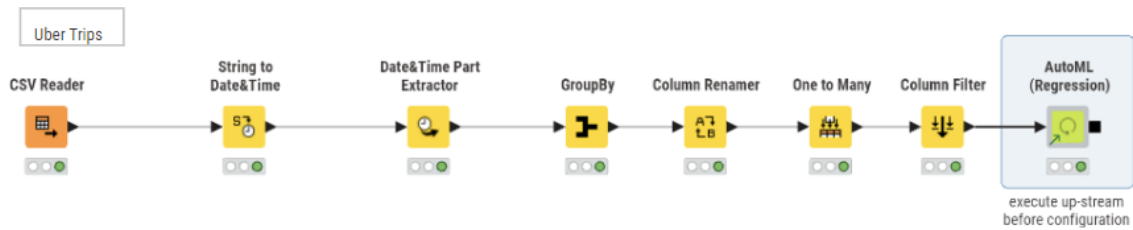
- Veri seti **1 milyondan fazla yolculuk kaydı** içermektedir.
- Tarih-saat, coğrafi koordinatlar ve kategorik base bilgileri bulunmaktadır.
- Veri setinde eksik (missing) değer bulunmamaktadır.
- Ham veri satır bazında tutulmuş olup, her satır bir Uber yolculuğunu temsil etmektedir.

Bu özellikler, veri setini zaman serisi tabanlı talep tahmin problemleri için uygun hâle getirmektedir.

• VERİ ÖN İŞLEME (Preprocessing)

Modelleme aşamasından önce veri seti üzerinde çeşitli veri ön işleme adımları uygulanmıştır. Tüm işlemler KNIME Analytics Platform üzerinde gerçekleştirilmiştir.

✓ KNIME Workflow Yapısı



Şekil 13 Uber Trips veri seti için oluşturulan KNIME workflow yapısı

Veri ön işleme süreci aşağıdaki temel adımlardan oluşmaktadır:

- Veri okuma
 - Tarih-saat dönüşümü
 - Zaman tabanlı özellik çıkarımı
 - Saatlik talep değişkeninin oluşturulması
 - Kategorik değişkenlerin sayısal forma dönüştürülmesi
-
- **Kullanılan Ön İşleme Adımları**
 - ✓ **CSV Reader:**

Uber yolculuk verileri KNIME ortamına aktarılmıştır.

- ✓ **String to Date&Time:**

Tarih-saat bilgileri string formatından Date&Time tipine dönüştürülmüştür.

- ✓ **Extract Date&Time Fields:**

Date&Time alanından aşağıdaki zaman tabanlı özellikler türetilmiştir:

- Gün (day)
- Saat (hour)
- Hafta günü (weekday)

- ✓ **GroupBy:**

Ham veri, gün, saat, hafta günü ve base bilgilerine göre gruplanmış; her grup için yolculuk sayısı hesaplanarak **saatlik talep değişkeni (trip_count)** elde edilmiştir.

- ✓ **One to Many:**

Base değişkeni, makine öğrenmesi algoritmalarında kullanılabilmesi için one-hot encoding yöntemiyle sayısal forma dönüştürülmüştür.

- ✓ **Column Filter:**

Modelleme sürecinde kullanılmayan veya bilgi tekrarına neden olan sütunlar veri setinden çıkarılmıştır.

- **MODELLEME VE AUTOML**

- ✓ **AutoML Yaklaşımı**

Bu çalışmada modelleme süreci, **KNIME AutoML (Regression)** node'u kullanılarak gerçekleştirilmiştir. AutoML yaklaşımı sayesinde farklı regresyon algoritmaları otomatik olarak eğitilmiş, hiperparametre ayarlamaları yapılmış ve modeller performanslarına göre karşılaştırılmıştır.

Model seçimi sürecinde performans metriği olarak **Root Mean Squared Error (RMSE)** kullanılmıştır.

- **Kullanılan Modeller**

AutoML süreci kapsamında aşağıdaki regresyon modelleri eğitilmiş ve değerlendirilmiştir:

- Linear Regression
- Polynomial Regression
- Regression Tree
- Random Forest
- Gradient Boosted Trees
- H2O Generalized Linear Model
- XGBoost Linear Ensemble
- XGBoost Tree Ensemble

Bu modeller, hem doğrusal hem de doğrusal olmayan yaklaşımları kapsayacak şekilde seçilmiştir.

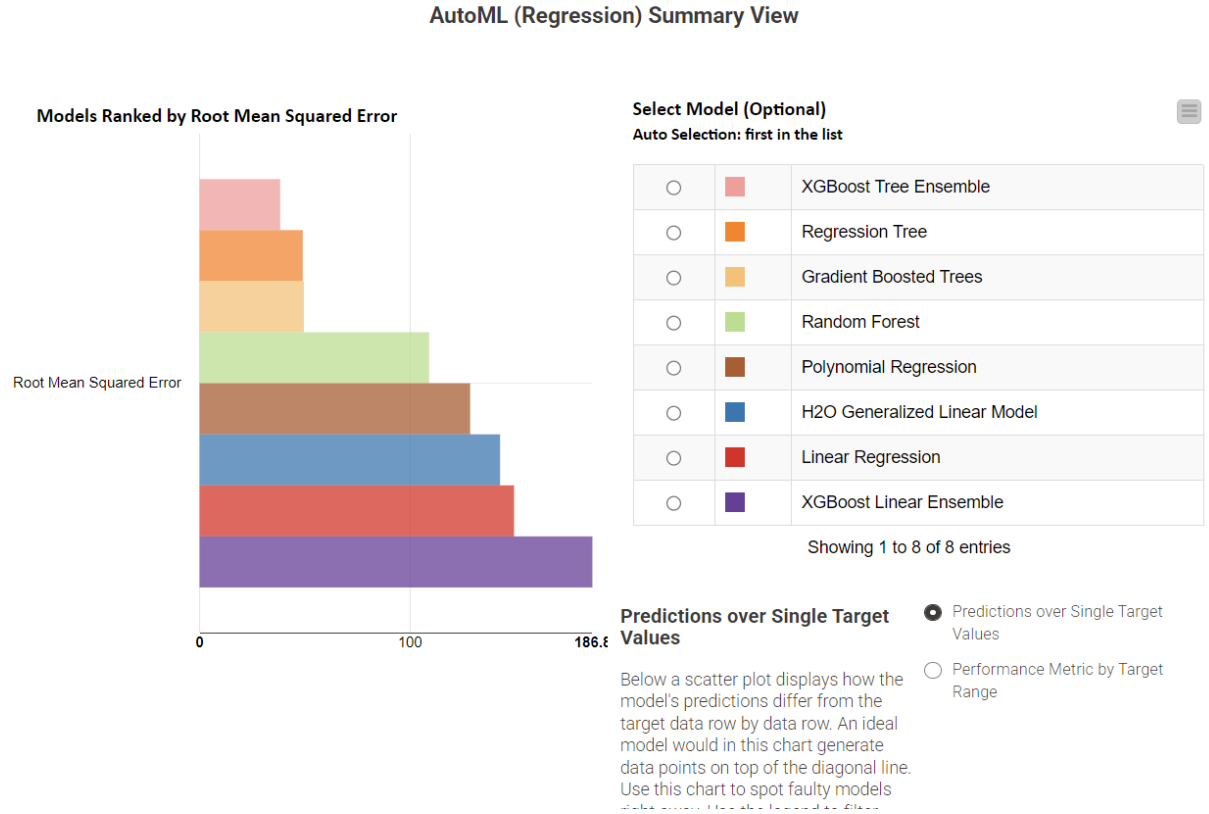
- **MODEL PERFORMANSI VE KARŞILAŞTIRMA**

- ✓ **Performans Metriği**

Model performanslarının değerlendirilmesinde Root Mean Squared Error (RMSE) metriği kullanılmıştır. RMSE, tahmin edilen değerler ile gerçek değerler arasındaki farkların büyüklüğünü ölçen bir metriktir ve büyük hataları daha fazla cezalandırmaktadır.

✓ Model Karşılaştırması

AutoML sonuçlarına göre eğitilen modeller RMSE değerlerine göre sıralanmıştır. Yapılan karşılaştırma sonucunda **XGBoost Tree Ensemble** modelinin en düşük RMSE değerine sahip olduğu ve diğer modellere kıyasla daha başarılı sonuçlar ürettiği görülmüştür. Bu nedenle nihai model olarak XGBoost Tree Ensemble seçilmiştir.



Şekil 14 Uber Trips veri seti için AutoML model karşılaştırması

• TAHMİN SONUÇLARININ GÖRSEL ANALİZİ

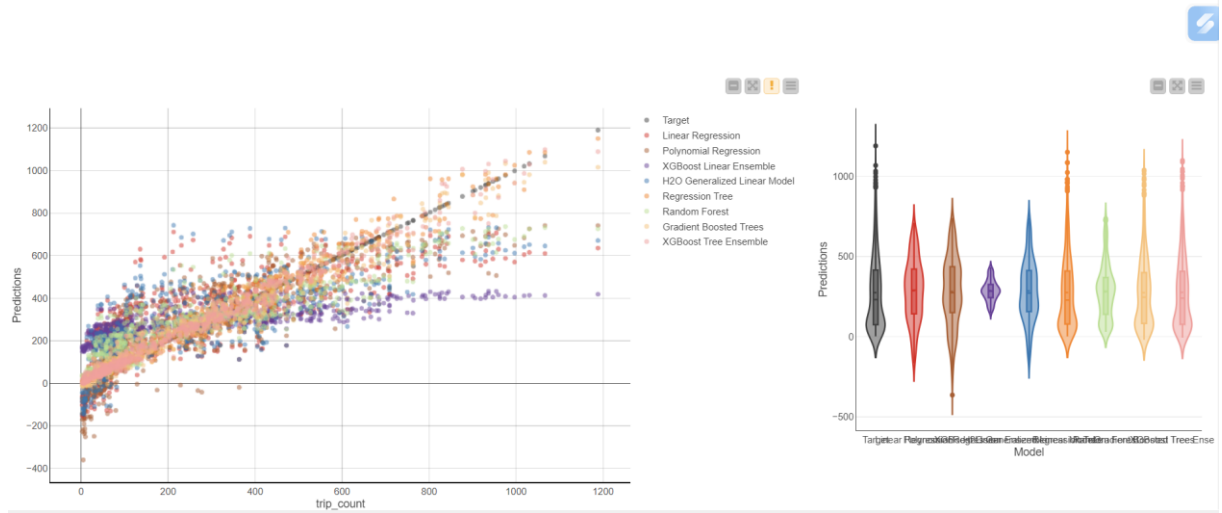
Model sonuçlarının daha iyi anlaşılabilmesi için tahmin değerleri görsel olarak analiz edilmiştir.

✓ Gerçek Değer – Tahmin Değeri Dağılım Grafiği:

Scatter grafikler, model tahminlerinin düşük ve orta yoğunluktaki saatlerde gerçek değerlere daha yakın olduğunu göstermektedir. Ancak yüksek talep içeren saatlerde hata payının arttığı gözlemlenmiştir.

✓ Dağılım (Violin) Grafikleri:

Saatlik yolculuk talebinin sağa çarpık bir dağılım sergilediği ve yoğun saatlerin sınırlı sayıda olduğu görülmektedir. Bu durum, yüksek talep saatlerinde model hatalarının artmasına neden olmaktadır.



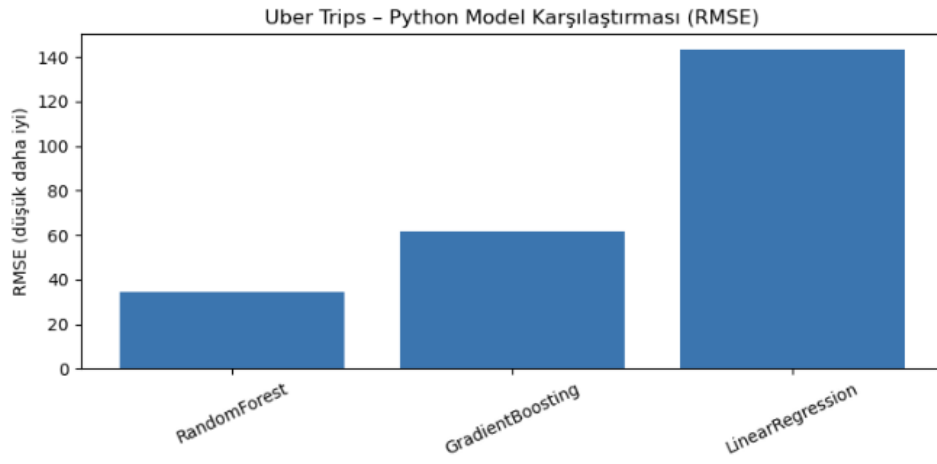
Şekil 15 Uber Trips veri seti için tahmin sonuçlarının görsel analizi

• Python (Jupyter Notebook) Ortamında Modelleme ve KNIME ile Karşılaştırma

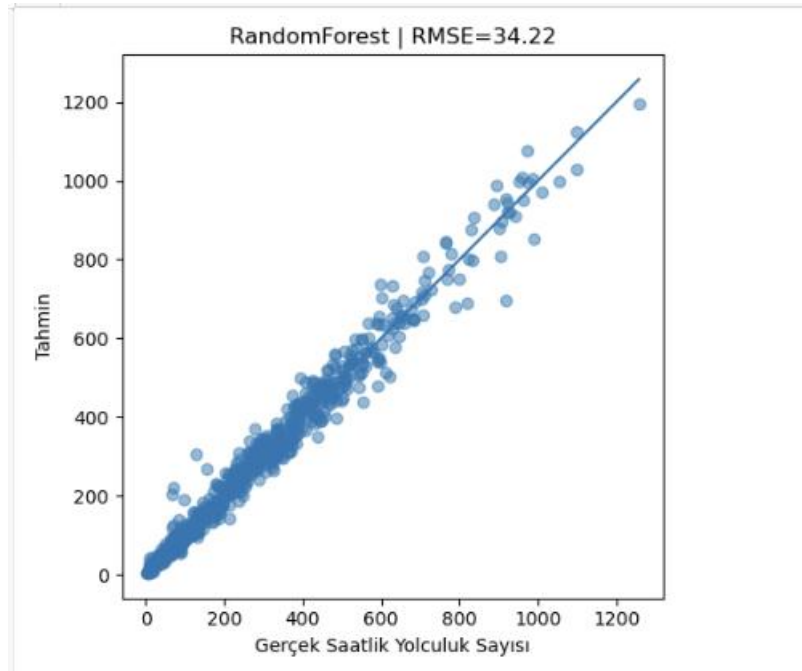
Uber Trips veri seti üzerinde gerçekleştirilen KNIME tabanlı analizlere ek olarak, aynı problem Python (Jupyter Notebook) ortamında da ele alınmıştır. Python tarafında, KNIME workflow'unda uygulanan veri ön işleme adımlarıyla uyumlu olacak şekilde tarih-saat dönüşümü yapılmış, zaman tabanlı özellikler (gün, saat, hafta günü) çıkarılmış ve GroupBy işlemi ile saatlik yolculuk talebi (trip_count) oluşturulmuştur. Base değişkeni One-Hot Encoding yöntemiyle sayısal forma dönüştürülmüştür.

Python ortamında Linear Regression, Random Forest ve Gradient Boosting modelleri karşılaştırılmış; performans değerlendirmesi RMSE metriği üzerinden yapılmıştır. Elde edilen sonuçlara göre **Random Forest** modeli RMSE = **34.23** değeri ile en başarılı model olarak belirlenmiştir. KNIME tarafında ise **XGBoost Tree Ensemble** modeli en düşük RMSE değerine sahip model olarak seçilmiştir. Her iki platformda da ağaç tabanlı ve doğrusal olmayan modellerin doğrusal modellere kıyasla daha başarılı sonuçlar üretmesi, Uber yolculuk talebinin zamana bağlı ve karmaşık bir yapıya sahip olduğunu göstermektedir.

C



Şekil 16 Python Ortamında Model Karşılaştırması (Uber Trip)



Şekil 17 Rabdom Forest Tahmin Performansı

Tablo 3 Uber Trips Veri Seti için KNIME ve Python Sonuçlarının Karşılaştırılması

<i>Kriter</i>	<i>KNIME Analytics Platform</i>	<i>Python (Jupyter Notebook)</i>
<i>Kullanılan Ortam</i>	KNIME AutoML (Regression)	Python (scikit-learn)
<i>Veri Ön İşleme</i>	Date&Time dönüşümü, GroupBy, One-Hot Encoding	DateTime parsing, GroupBy, One-Hot Encoding
<i>Eğitim–Test Bölünmesi</i>	AutoML varsayılan ayarları	%80 – %20, random_state = 42
<i>Kullanılan Modeller</i>	Linear, Tree, Random Forest, Gradient Boosted Trees, XGBoost	Linear Regression, Random Forest, Gradient Boosting
<i>Performans Metriği</i>	RMSE	RMSE
<i>En İyi Model</i>	XGBoost Tree Ensemble	Random Forest Regressor
<i>En İyi RMSE</i>	En düşük RMSE (AutoML sonucu)	34.23
<i>Model Ailesi</i>	Ağaç tabanlı / Boosting	Ağaç tabanlı
<i>Genel Yorum</i>	Doğrusal olmayan ve zamana bağlı yapı	Doğrusal olmayan ve zamana bağlı yapı

Tablo 3’te, Uber Trips veri seti üzerinde gerçekleştirilen regresyon analizlerinin KNIME Analytics Platform ve Python (Jupyter Notebook) ortamlarındaki sonuçları karşılaştırmalı olarak sunulmaktadır. Her iki ortamda da tarih-saat dönüşümü, zaman tabanlı özellik çıkarımı ve saatlik talep değişkeninin oluşturulması gibi benzer veri ön işleme adımları uygulanmıştır. Sonuçlar incelendiğinde, doğrusal modellerin yetersiz kaldığı; buna karşılık ağaç tabanlı yöntemlerin daha düşük RMSE değerleri ürettiği görülmektedir. KNIME tarafında XGBoost Tree Ensemble, Python tarafında ise Random Forest modeli en başarılı sonuçları vermiştir. Bu durum, Uber yolculuk talebinin doğrusal olmayan ve zamana bağlı karmaşık bir yapıya sahip olduğunu göstermektedir.

- **SONUÇ VE DEĞERLENDİRME**

Bu çalışmada, Uber Trips veri seti kullanılarak saatlik yolculuk talebinin tahmin edilmesine yönelik bir regresyon problemi hem **KNIME Analytics Platform** hem de **Python (Jupyter Notebook)** ortamında ele alınmıştır. Tarih-saat dönüşümü ve zaman tabanlı özellik çıkarımı adımlarının ardından, KNIME tarafında AutoML yaklaşımı ile birden fazla model eğitilmiş ve RMSE metriğine göre karşılaştırılmıştır. Yapılan değerlendirme sonucunda **XGBoost Tree Ensemble** modeli en başarılı model olarak belirlenmiştir.

Aynı problem Python ortamında manuel olarak ele alınmış; Linear Regression, Random Forest ve Gradient Boosting modelleri karşılaştırılmıştır. Python sonuçlarına göre **Random Forest** modeli $RMSE = 34.23$ değeri ile en iyi performansı göstermiştir. Doğrusal regresyon modelinin yüksek hata üretmesi, problemin doğrusal olmayan bir yapıya sahip olduğunu açıkça ortaya koymaktadır.

Her iki platformda elde edilen sonuçlar birlikte değerlendirildiğinde, Uber yolculuk talebinin zamana bağlı dalgalanmalar ve karmaşık ilişkiler içerdiği, bu nedenle ağaç tabanlı ve boosting yöntemlerinin daha uygun olduğu görülmektedir. AutoML yaklaşımı hızlı ve etkili model seçimi sağlarken, Python ortamı model davranışlarının daha ayrıntılı analiz edilmesine olanak tanımıştır.

4. CREDIT CARD CLUSTERING – MAKİNE ÖĞRENMESİ UYGULAMASI

• PROBLEM TANIMI

Bu çalışmada amaç, kredi kartı kullanıcılarını harcama alışkanlıkları, bakiye düzeyleri ve nakit avans kullanım davranışlarına göre anlamlı müşteri segmentlerine ayırmaktır. Müşteri segmentasyonu, bankalar ve finansal kuruluşlar için risk yönetimi, hedefli pazarlama ve müşteri ilişkileri stratejilerinin geliştirilmesinde önemli bir rol oynamaktadır. Bu kapsamda, gözetimsiz öğrenme yöntemlerinden biri olan kümeleme algoritmaları kullanılarak kredi kartı kullanıcılarının benzer davranış örüntülerine göre gruplandırılması hedeflenmiştir.

• VERİ SETİ TANITIMI

Bu çalışmada Kaggle platformunda yer alan **Credit Card Dataset (CC GENERAL)** veri seti kullanılmıştır. Veri seti, kredi kartı kullanıcılarına ait çeşitli finansal ve davranışsal özellikleri içermektedir. Toplamda binlerce gözlemden oluşan veri seti, kullanıcıların bakiye durumu, alışveriş miktarları, taksitli ve tek seferlik harcamaları, nakit avans kullanımı ve kart kullanım sıklığı gibi değişkenleri kapsamaktadır.

Veri setinde yer alan başlıca değişkenler şunlardır:

- **BALANCE**: Kredi kartı bakiyesi
- **PURCHASES**: Toplam alışveriş tutarı
- **ONEOFF_PURCHASES**: Tek seferlik alışverişler
- **INSTALLMENTS_PURCHASES**: Taksitli alışverişler
- **CASH_ADVANCE**: Nakit avans tutarı
- **PURCHASES_FREQUENCY**: Alışveriş sıklığı
- **CREDIT_LIMIT**: Kredi kartı limiti
- **PAYMENTS**: Yapılan ödemeler

- **VERİ ÖN İŞLEME (PREPROCESSING)**

Kümeleme algoritmalarının mesafe tabanlı yapısı nedeniyle veri ön işleme adımı büyük önem taşımaktadır. Bu kapsamda, veri setinde yer alan tüm sayısal değişkenler **Z-score normalizasyonu** ile ölçeklendirilmiştir. Bu yöntem ile her bir değişkenin ortalaması sıfır, standart sapması bir olacak şekilde dönüştürülmesi sağlanmış ve farklı ölçeklere sahip değişkenlerin kümeleme sürecini olumsuz etkilemesi önlenmiştir.

Normalizasyon işlemi sırasında orijinal değerler yerine normalize edilmiş değerler kullanılarak, K-Means algoritmasının tüm değişkenleri eşit önemde değerlendirmesi sağlanmıştır.

- **MODELLEME – K-MEANS KÜMELEME**

Veri seti üzerinde gözetimsiz öğrenme yöntemlerinden **K-Means kümeleme algoritması** uygulanmıştır. Kümeleme işlemi, Z-score ile normalize edilmiş tüm sayısal değişkenler kullanılarak gerçekleştirilmiştir. Küme sayısı, kredi kartı müşteri segmentasyonu literatüründe yaygın olarak kullanılan değerler dikkate alınarak **k = 4** olarak belirlenmiştir.

Algoritmanın kararlı ve tekrarlanabilir sonuçlar üretmesi amacıyla küme merkezleri **“first k rows”** yöntemi ile başlatılmış ve sabit rastgele tohum (random seed) kullanılmıştır.

- **KÜMELEME SONUÇLARI VE CLUSTER ANALİZİ**

K-Means algoritması sonucunda kredi kartı kullanıcıları dört farklı kümeye ayrılmıştır. Küme merkezleri incelendiğinde, her bir kümenin belirgin finansal ve davranışsal özelliklere sahip olduğu gözlemlenmiştir.

- ✓ **Cluster 0 – Düşük Aktivite / Dengeli Kullanıcılar**

Bu küme, kredi kartını düşük sıklıkta kullanan, alışveriş ve nakit avans tutarları ortalamanın altında olan kullanıcıları temsil etmektedir. Düşük riskli ve pasif müşteri profili olarak değerlendirilebilir.

✓ **Cluster 1 – Nakit Avans Ağırlıklı / Riskli Kullanıcılar**

Bu küme, yüksek bakiye ve yüksek nakit avans kullanımına sahip kullanıcıları içermektedir. Tam ödeme oranının düşük olması ve nakit avans kullanımının yoğunluğu, bu grubun finansal risk açısından dikkat edilmesi gereken bir müşteri segmenti olduğunu göstermektedir.

✓ **Cluster 2 – Aktif Alışveriş Yapan Değerli Kullanıcılar**

Bu kümede yer alan kullanıcılar, alışveriş tutarı ve alışveriş sıklığı açısından ortalamanın üzerinde değerlere sahiptir. Taksitli ve tek seferlik harcamaların yüksek olması, bu grubun aktif ve yüksek değerli müşteri segmentini temsil ettiğini göstermektedir. Nakit avans kullanımının düşük olması, kontrollü kullanım davranışını desteklemektedir.

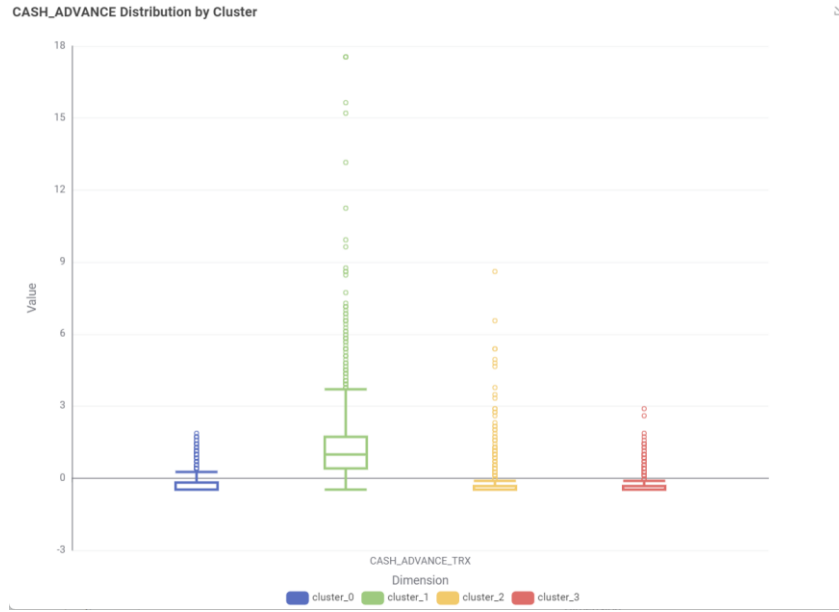
✓ **Cluster 3 – Düşük Bakiye / Düşük Kullanım Profili**

Bu küme, kredi kartını nadiren kullanan, bakiye ve harcama tutarları düşük olan kullanıcıları kapsamaktadır. Kart kullanım alışkanlığı zayıf olan bu grup, potansiyel olarak pasif müşteri segmenti olarak değerlendirilebilir.

• **GÖRSEL ANALİZ (BOX PLOT VE DAĞILIM GRAFİKLERİ)**

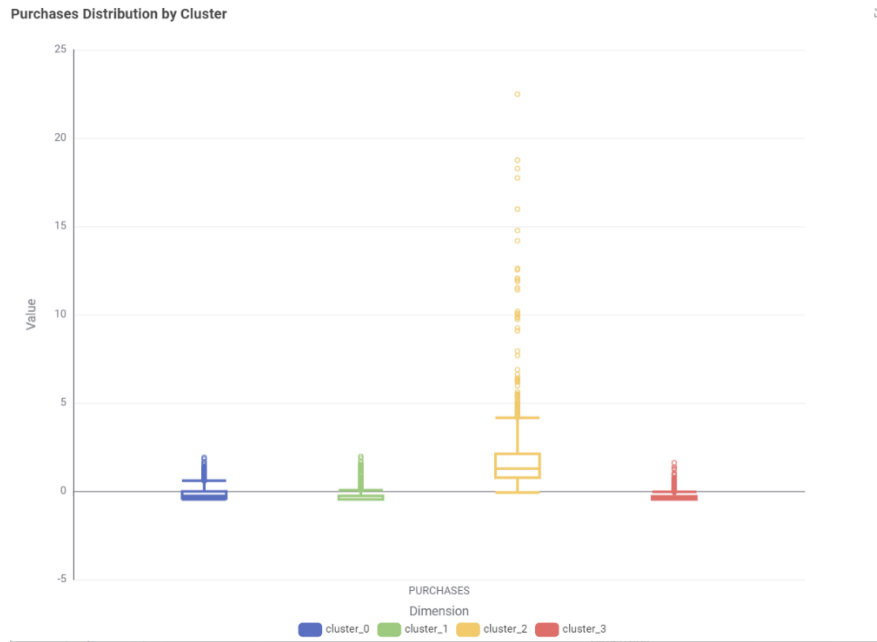
Kümeleme sonuçlarının daha iyi anlaşılabilmesi amacıyla, çeşitli görselleştirme teknikleri kullanılmıştır. Box plot analizleri, cluster'lar arasında bakiye, alışveriş ve nakit avans dağılımlarının belirgin şekilde farklılaştığını göstermektedir. Özellikle nakit avans kullanımına ilişkin görseller, riskli kullanıcı grubunun diğer kümelerden net bir şekilde ayrıştığını ortaya koymaktadır.

Bu görseller, kümeleme sonuçlarının yalnızca sayısal değil, aynı zamanda görsel olarak da doğrulanmasını sağlamıştır.



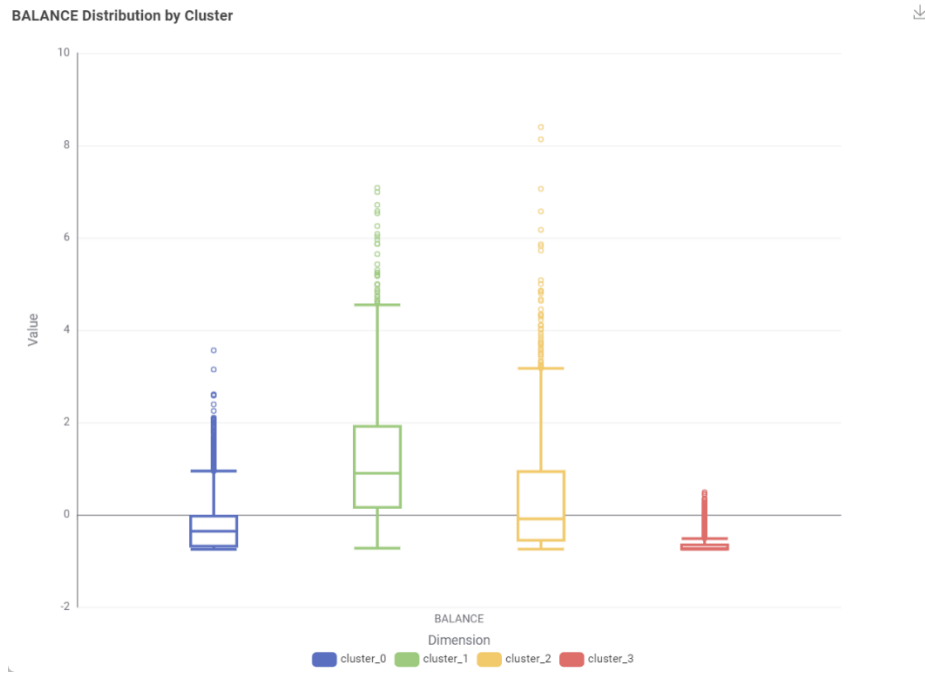
Şekil 18 CASH ADVANCE Değerlerinin Cluster Bazında Dağılımı

Nakit avans kullanımına ait dağılımlar incelendiğinde, **Cluster 1**'in diğer cluster'lara kıyasla belirgin şekilde daha yüksek nakit avans değerlerine sahip olduğu görülmektedir. Buna karşılık Cluster 0, Cluster 2 ve Cluster 3'te nakit avans kullanımının düşük seviyelerde kaldığı gözlemlenmiştir. Bu durum, Cluster 1'in daha riskli kredi kartı kullanım profiline sahip olduğunu açıkça göstermektedir.



Şekil 19 PURCHASES Değerlerinin Cluster Bazında Dağılımı

Alışveriş tutarlarına ilişkin box plot analizi, **Cluster 2**'nin diğer cluster'lara kıyasla belirgin şekilde daha yüksek alışveriş değerlerine sahip olduğunu ortaya koymaktadır. Cluster 0, Cluster 1 ve Cluster 3'te alışveriş seviyelerinin görece düşük olduğu görülmektedir. Bu sonuçlar, Cluster 2'nin aktif ve yüksek değerli müşteri segmentini temsil ettiğini desteklemektedir.



Şekil 20 BALANCE Değerlerinin Cluster Bazında Dağılımı

Box plot sonuçları, cluster'lar arasında bakiye seviyelerinin belirgin biçimde farklılaştığını göstermektedir. Özellikle **Cluster 1**'in pozitif yönde yüksek bakiye değerlerine sahip olduğu, **Cluster 0** ve **Cluster 3**'ün ise ortalamanın altında bakiye seviyeleri ile ayrıştığı gözlemlenmiştir. Bu durum, Cluster 1'in finansal risk açısından daha dikkat edilmesi gereken bir kullanıcı grubunu temsil ettiğini göstermektedir.

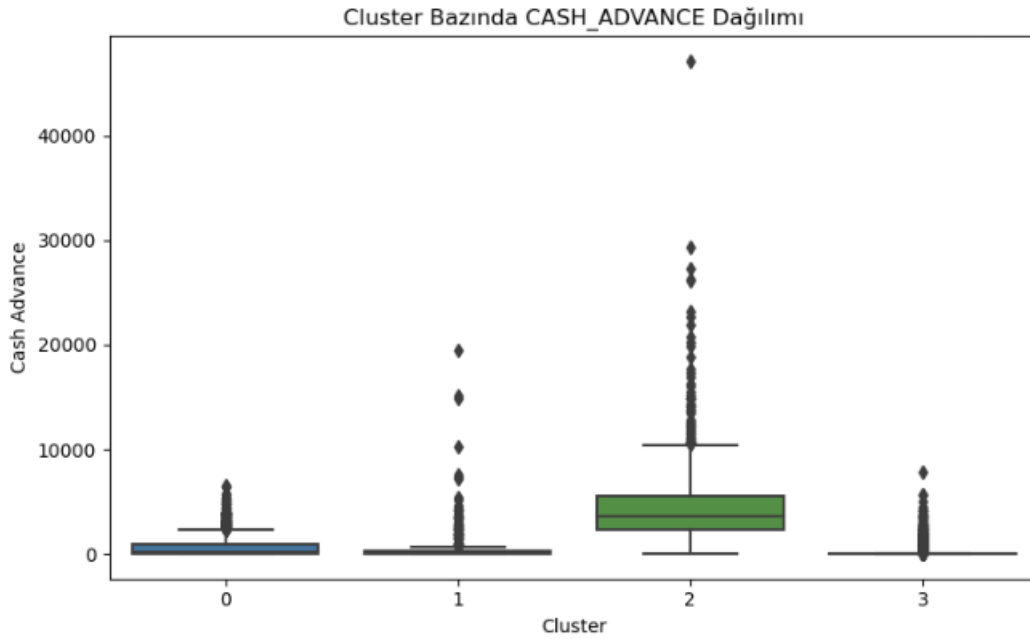
- **Python (Jupyter Notebook) Ortamında Modelleme ve KNIME ile Karşılaştırma**

Credit Card (CC GENERAL) veri seti üzerinde gerçekleştirilen kümeleme analizi, Python (Jupyter Notebook) ortamında K-Means algoritması kullanılarak tekrar edilmiştir. Kümeleme öncesinde, KNIME tarafında olduğu gibi tüm sayısal değişkenler Z-score normalizasyonu ile ölçeklendirilmiş ve küme sayısı $k = 4$ olarak belirlenmiştir.

Python ortamında elde edilen kümeleme sonucunda silhouette score değeri **0.197** olarak hesaplanmıştır. Bu değer, kümeler arasında net bir geometrik ayrışma olmadığını ancak rastgele bir kümeleme de söz konusu olmadığını göstermektedir. Finansal ve davranışsal özellikler

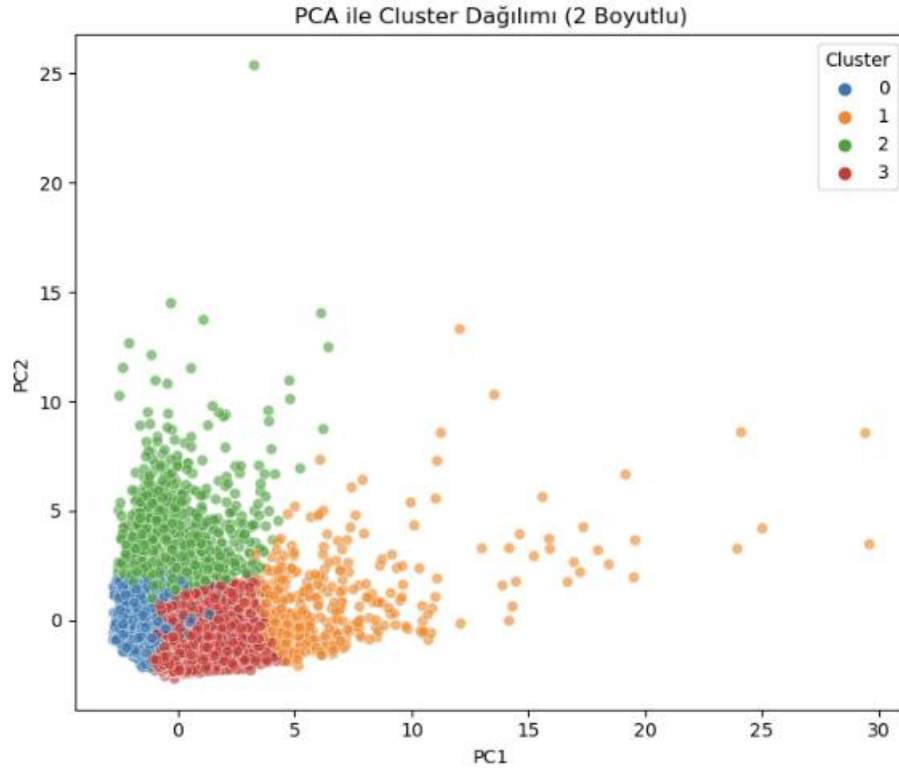
içeren gerçek hayat veri setlerinde, özellikle müşteri segmentasyonu problemlerinde, bu tür silhouette değerleri literatürde yaygın olarak görülmektedir.

Küme merkezleri incelendiğinde, her bir kümenin belirgin harcama, bakiye ve nakit avans davranışları sergilediği gözlemlenmiştir. Bu durum, matematiksel ayrışma sınırlı olsa dahi, elde edilen kümelerin işlevsel ve yorumlanabilir müşteri segmentlerini temsil ettiğini göstermektedir. Python ve KNIME ortamlarında elde edilen kümeleme sonuçları, küme sayısı ve müşteri profilleri açısından birbiriyle tutarlı bulunmuştur.



Şekil 21 Cluster Bazında CASH_ADVANCE Dağılımı

Şekil 21’de, CASH_ADVANCE değişkeninin cluster bazında dağılımı gösterilmektedir. Grafik incelendiğinde, Cluster 2’nin diğer cluster’lara kıyasla belirgin şekilde daha yüksek nakit avans kullanımına sahip olduğu görülmektedir. Bu durum, ilgili cluster’ın daha riskli kredi kartı kullanım profiline sahip olduğunu göstermektedir.



Şekil 22 PCA ile Cluster Dağılımı (2 Boyutlu)

PCA ile iki boyuta indirgenen veri üzerinde cluster dağılımı incelendiğinde, kümelerin tamamen net sınırlarla ayrılmadığı ancak belirli yoğunluk bölgelerinde toplandığı görülmektedir. Bu durum, silhouette score değerinin görece düşük olmasını açıklamakla birlikte, elde edilen kümelerin davranışsal olarak anlamlı olduğunu göstermektedir.

Tablo 4 Credit Card (CC GENERAL) Veri Seti için KNIME ve Python Sonuçlarının Karşılaştırılması

<i>Kriter</i>	<i>KNIME Analytics Platform</i>	<i>Python (Jupyter Notebook)</i>
<i>Kullanılan Ortam</i>	KNIME	Python (scikit-learn)
<i>Problem Türü</i>	Gözetimsiz Öğrenme (Clustering)	Gözetimsiz Öğrenme (Clustering)
<i>Veri Ön İşleme</i>	Missing Value (Median), Z-Score Normalizasyon	Missing Value (Median), StandardScaler (Z-Score)
<i>Kullanılan Algoritma</i>	K-Means	K-Means
<i>Küme Sayısı (k)</i>	4	4
<i>Performans Ölçütü</i>	Görsel analiz (Box Plot, dağılım)	Silhouette Score
<i>Silhouette Score</i>	Görsel değerlendirme	0.197
<i>Küme Profilleri</i>	Harcama, bakiye ve nakit avans bazlı ayırım	Harcama, bakiye ve nakit avans bazlı ayırım
<i>Genel Yorum</i>	Davranışsal olarak anlamlı kümeler	Davranışsal olarak anlamlı ancak geometrik ayrışma sınırlı

Tablo 4’te, Credit Card (CC GENERAL) veri seti üzerinde gerçekleştirilen kümeleme analizlerinin KNIME ve Python ortamlarındaki sonuçları karşılaştırılmaktadır. Her iki platformda da benzer veri ön işleme adımları ve aynı küme sayısı kullanılarak elde edilen sonuçlar, müşteri segmentlerinin davranışsal olarak tutarlı biçimde ayrıldığını göstermektedir.

- **SONUÇ VE DEĞERLENDİRME**

Bu çalışmada, Credit Card (CC GENERAL) veri seti kullanılarak kredi kartı kullanıcılarının harcama, bakiye ve nakit avans davranışlarına göre segmentlere ayrılması amaçlanmıştır. Kümeleme analizi hem **KNIME Analytics Platform** hem de **Python (Jupyter Notebook)** ortamında gerçekleştirilmiş ve her iki platformda da **K-Means** algoritması kullanılmıştır. Kümeleme öncesinde eksik değerler medyan ile doldurulmuş ve tüm sayısal değişkenler Z-score normalizasyonu ile ölçeklendirilmiştir.

Python ortamında gerçekleştirilen kümeleme sonucunda silhouette score değeri **0.197** olarak hesaplanmıştır. Bu değer, kümeler arasında net bir geometrik ayrışma olmadığını ancak tamamen rastgele bir kümelenmenin de söz konusu olmadığını göstermektedir. Gerçek hayat finansal veri setlerinde, özellikle müşteri davranışlarını içeren problemlerde bu tür silhouette değerleri literatürde yaygın olarak görülmektedir.

Küme merkezleri ve görsel analizler incelendiğinde, her bir kümenin farklı harcama ve kredi kartı kullanım profilleri sergilediği gözlemlenmiştir. Bazı kümeler yüksek nakit avans ve bakiye değerleri ile daha riskli müşteri profillerini temsil ederken, bazı kümeler düzenli alışveriş yapan ve düşük riskli kullanıcıları göstermektedir. Bu durum, matematiksel ayrışma sınırlı olsa dahi elde edilen kümelerin **işlevsel ve yorumlanabilir müşteri segmentleri** sunduğunu ortaya koymaktadır.

Sonuç olarak, KNIME ve Python ortamlarında elde edilen kümeleme sonuçları birbiriyle tutarlı olup, kullanılan araçlar farklı olsa da ortaya çıkan müşteri profilleri benzer özellikler göstermektedir. Bu çalışma, kümeleme analizlerinde yalnızca sayısal performans ölçütlerinin değil, aynı zamanda iş alanına yönelik yorumlanabilirliğin de önemli olduğunu göstermektedir.

5. YouTube Yorumlarında Spam Tespiti

Sosyal medya platformlarında kullanıcı etkileşiminin artmasıyla birlikte spam içerikli yorumlar da ciddi bir problem haline gelmiştir. Bu tür yorumlar; reklam, yönlendirme veya alakasız içerikler barındırarak kullanıcı deneyimini olumsuz etkilemektedir. Bu çalışmada, YouTube videolarına ait yorumlar kullanılarak bir yorumun **spam (1)** veya **spam olmayan (0)** olup olmadığının otomatik olarak tespit edilmesi amaçlanmıştır.

- **Veri Seti**

Çalışmada Kaggle üzerinde paylaşılan ve farklı YouTube videolarına ait yorumları içeren veri seti kullanılmıştır. Veri seti birden fazla CSV dosyasından oluşmakta olup, KNIME ortamında **Concatenate** node'u kullanılarak tek bir tablo haline getirilmiştir.

Veri setinde yer alan temel sütunlar aşağıdaki gibidir:

- **CONTENT:** Kullanıcı tarafından yazılan yorum metni
- **CLASS:** Yorumun etiketi (1: Spam, 0: Spam değil)

Toplamda yaklaşık **2000 civarında yorum** bulunmaktadır ve sınıflar arasında kısmi bir dengesizlik mevcuttur.

- **Veri Ön İşleme (Text Preprocessing)**

Makine öğrenmesi modellerinin metin verisi üzerinde daha etkili çalışabilmesi için çeşitli ön işleme adımları uygulanmıştır:

- Yorum metinleri küçük harfe dönüştürülmüştür
- Noktalama işaretleri ve sayılar temizlenmiştir
- Gereksiz boşluklar normalize edilmiştir

Bu işlemler **Text Preprocessing** node'u kullanılarak gerçekleştirilmiştir.

- **Özellik Çıkarımı (Feature Extraction)**

Ön işlenmiş metinler sayısal forma dönüştürülmüştür. Bu amaçla aşağıdaki adımlar izlenmiştir:

1. **Strings to Document:** Metinlerin doküman yapısına dönüştürülmesi
2. **Bag of Words Creator:** Kelime temelli temsil oluşturulması
3. **TF (Term Frequency)** ve **IDF (Inverse Document Frequency)** hesaplanması
4. **Document Vector:** TF-IDF tabanlı belge vektörlerinin elde edilmesi

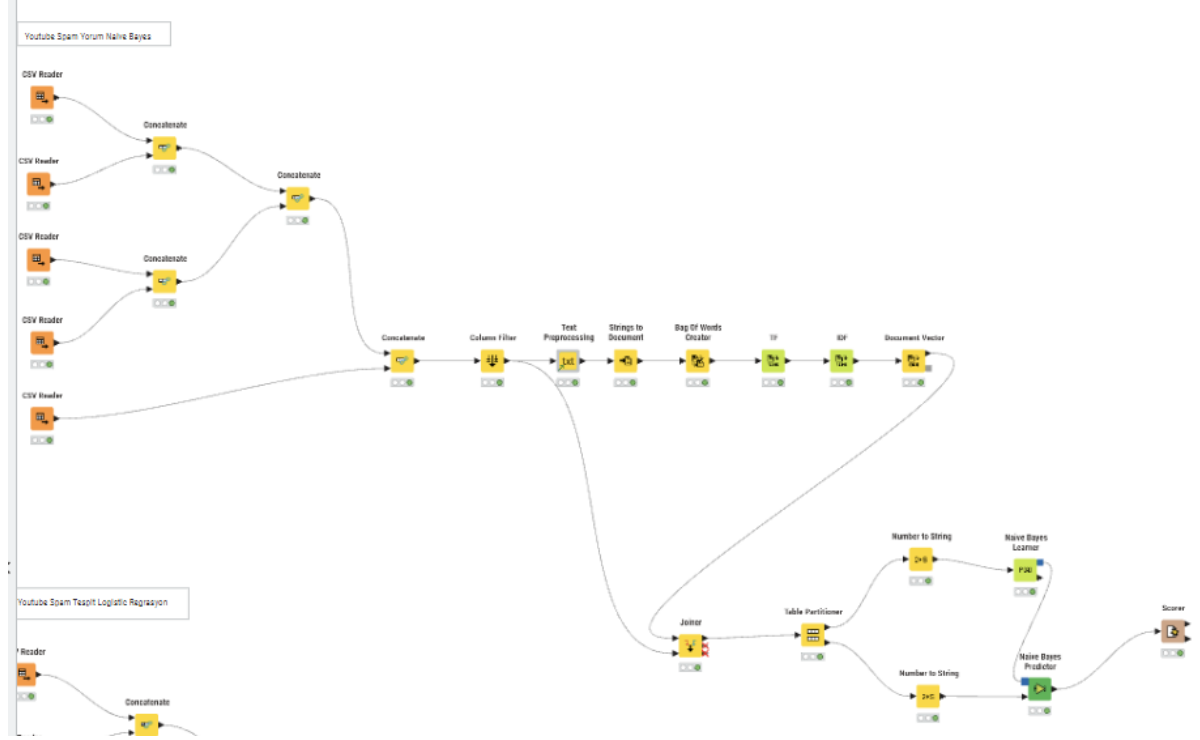
Bu aşama sonucunda her yorum, yüksek boyutlu bir **sayısal vektör** ile temsil edilmiştir.

- **Modelleme ve Eğitim**

Veri seti **%80 eğitim – %20 test** olacak şekilde **Table Partitioner** node’u ile ikiye ayrılmıştır. Çalışma kapsamında iki farklı sınıflandırma algoritması kullanılmıştır.

- ✓ **Naive Bayes Sınıflandırıcı**

İlk model olarak **Naive Bayes** algoritması tercih edilmiştir. Bu algoritma, kelimelerin bağımsız olduğu varsayımına dayanarak olasılık temelli bir sınıflandırma gerçekleştirir. Metin sınıflandırma problemlerinde sıkça kullanılan hızlı ve basit bir yöntemdir.



Şekil 23 Youtube Yorumları veri seti için oluşturulan Naive Bayes KNIME workflow yapısı

► 1: Confusion matrix		► 2: Accuracy statistics		Flow Variables	
Rows: 2 Columns: 2		Table Statistics			
#	RowID	1 No Number (integer)	0 No Number (integer)		
1	1	204	0		
2	0	185	0		

Şekil 24 Naive Bayes Konfüzyon Matrisi

- **Confusion Matrix (Karmaşıklık Matrisi) Yorumu**

Elde edilen confusion matrix sonuçlarına göre modelin sınıflandırma davranışı aşağıdaki gibidir:

- **True Positive (TP) = 204**

Gerçek sınıfı *pozitif* olan 204 yorum doğru şekilde pozitif olarak sınıflandırılmıştır.

- **False Positive (FP) = 185**

Gerçek sınıfı *negatif* olan 185 yorum, model tarafından yanlışlıkla pozitif olarak tahmin edilmiştir.

- **True Negative (TN) = 204**

Gerçek sınıfı *negatif* olan 204 yorum doğru şekilde negatif olarak sınıflandırılmıştır.

- **False Negative (FN) = 185**

Gerçek sınıfı *pozitif* olan 185 yorum, model tarafından yanlışlıkla negatif olarak tahmin edilmiştir.

Bu sonuçlar, modelin iki sınıf arasında **dengeli fakat ayırt ediciliği düşük** bir performans sergilediğini göstermektedir. Özellikle yanlış pozitif ve yanlış negatif değerlerinin yüksek olması, Naive Bayes modelinin YouTube yorumları gibi bağlama ve kelime ilişkilerine duyarlı metinlerde sınırlı kaldığını göstermektedir.

- **Accuracy (Doğruluk) ve Diğer Performans Metrikleri Yorumu**

Modelin performans ölçütleri aşağıda özetlenmiştir:

- ✓ **Accuracy (Doğruluk): 0.524**

Model, tüm yorumların yaklaşık %52.4'ünü doğru sınıflandırabilmiştir. Bu değer, modelin rastgele tahmine yakın bir başarı sergilediğini göstermektedir.

- ✓ **Precision (Kesinlik): 0.524**

Pozitif olarak tahmin edilen yorumların %52.4'ü gerçekten pozitiftir. Yanlış pozitif oranının yüksek olması, precision değerini düşürmüştür.

- ✓ **Recall / Sensitivity (Duyarlılık): 0.50**

Gerçek pozitif yorumların yalnızca %50'si doğru şekilde tespit edilebilmiştir. Bu durum, modelin pozitif sınıfı yakalamakta zorlandığını göstermektedir.

✓ **Specificity (Özgüllük): 0.50**

Negatif yorumların %50'si doğru şekilde tanımlanabilmiştir. Bu da negatif sınıf için ayırt ediciliğin düşük olduğunu göstermektedir.

✓ **F-Measure (F1-Score): 0.688**

Precision ve recall değerlerinin dengeli bir bileşimi olarak hesaplanan F1 skoru orta seviyededir ancak güçlü bir sınıflandırma performansı için yeterli değildir.

✓ **Cohen's Kappa: 0**

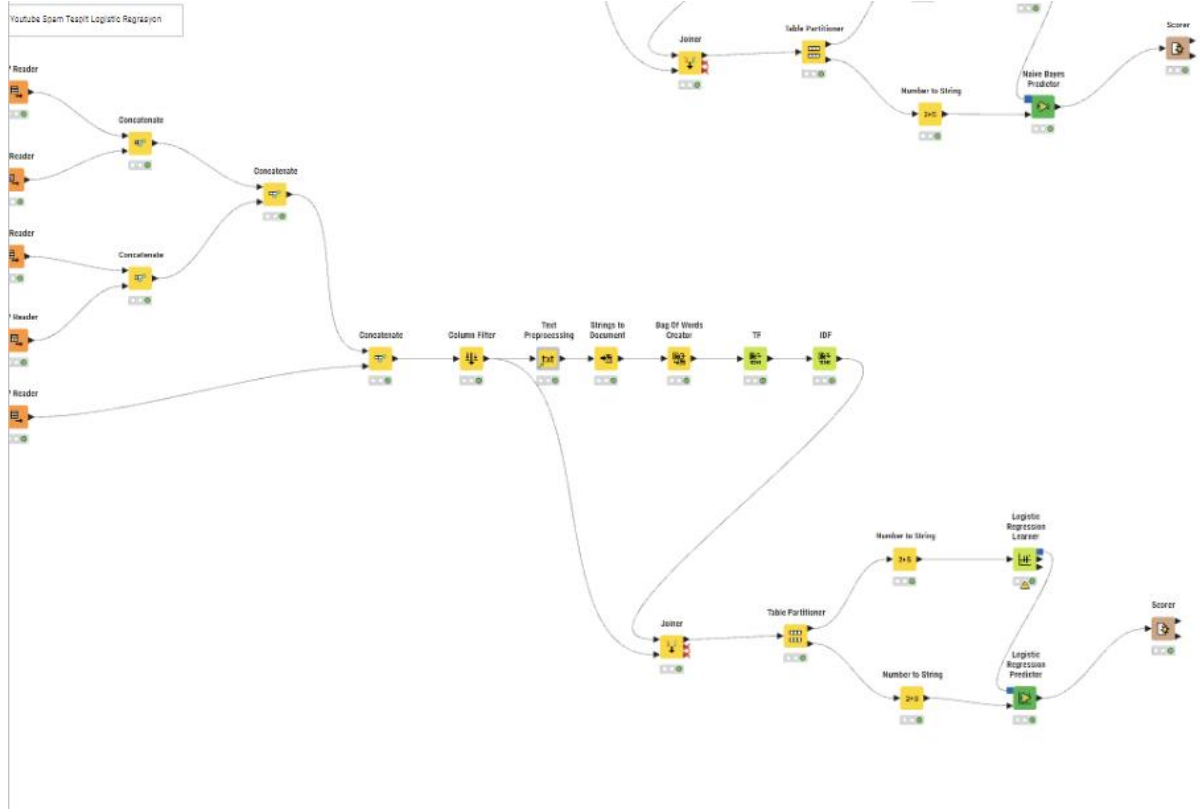
Kappa değerinin 0 olması, modelin sınıflandırma başarısının **rastgele tahminden anlamlı derecede farklı olmadığını** göstermektedir.

• **Genel Değerlendirme**

Naive Bayes algoritması ile YouTube yorumları veri seti üzerinde gerçekleştirilen sınıflandırma çalışmasında, modelin genel doğruluk oranı %52.4 olarak elde edilmiştir. Confusion matrix sonuçları incelendiğinde, yanlış pozitif ve yanlış negatif değerlerinin yüksek olduğu görülmüştür. Precision, recall ve specificity değerlerinin 0.5 civarında olması, modelin iki sınıf arasında yeterli ayırt ediciliğe sahip olmadığını göstermektedir. Cohen's Kappa değerinin 0 olması ise model performansının rastgele sınıflandırmaya yakın olduğunu ortaya koymaktadır. Bu sonuçlar, Naive Bayes algoritmasının YouTube yorumları gibi bağlamsal ve dilsel çeşitliliğin yüksek olduğu metin verilerinde sınırlı bir performans sunduğunu göstermektedir.

✓ Logistic Regression

İkinci model olarak **Logistic Regression** algoritması uygulanmıştır. Bu model, doğrusal bir karar sınırı oluşturarak yorumun spam olma olasılığını tahmin etmektedir. TF-IDF tabanlı vektörlerle birlikte kullanıldığında metin sınıflandırma problemlerinde etkili sonuçlar verebilmektedir.



Şekil 25 Youtube Yorumları veri seti için oluşturulan Logistic Regrasyon KNIME workflow yapısı

► 1: Confusion matrix ► 2: Accuracy statistics ☒ Flow Variables

Rows: 2 | Columns: 2

	#	RowID	1 Number (Integer)	0 Number (Integer)
☐	1	1	208	0
☐	2	0	184	0

Şekil 26 Logistic Regression Konfüzyon Matrisi

- **Confusion Matrix (Karmaşıklık Matrisi) Yorumu – Logistic Regression**

Elde edilen confusion matrix sonuçları şu şekildedir:

- ✓ **True Positive (TP) = 208**

Gerçek sınıfı *pozitif* olan 208 yorum doğru şekilde pozitif olarak sınıflandırılmıştır.

- ✓ **False Positive (FP) = 184**

Gerçek sınıfı *negatif* olan 184 yorum, model tarafından yanlışlıkla pozitif olarak tahmin edilmiştir.

- ✓ **True Negative (TN) = 208**

Gerçek sınıfı *negatif* olan 208 yorum doğru şekilde negatif olarak sınıflandırılmıştır.

- ✓ **False Negative (FN) = 184**

Gerçek sınıfı *pozitif* olan 184 yorum yanlışlıkla negatif olarak sınıflandırılmıştır.

Bu dağılım, Logistic Regression modelinin her iki sınıfta da **simetrik fakat sınırlı bir ayırt ediciliğe** sahip olduğunu göstermektedir. Yanlış sınıflandırmaların (FP ve FN) yüksek olması, modelin metin tabanlı duygusal ayrımı yeterince güçlü şekilde öğrenemediğini ortaya koymaktadır.

- **Accuracy ve Performans Metrikleri Yorumu**

Modelin hesaplanan performans ölçütleri aşağıda özetlenmiştir:

- ✓ **Accuracy (Doğruluk): 0.531**

Model, tüm yorumların %53.1'ini doğru sınıflandırabilmiştir. Bu değer, Naive Bayes modeline kıyasla **çok küçük bir iyileşme** göstermektedir.

- ✓ **Precision (Kesinlik): 0.531**

Pozitif olarak tahmin edilen yorumların %53.1'i gerçekten pozitiftir. Yanlış pozitif sayısının yüksek olması, precision değerini sınırlamaktadır.

- ✓ **Recall / Sensitivity (Duyarlılık): 0.50**

Gerçek pozitif yorumların yalnızca %50'si doğru şekilde tespit edilmiştir. Bu durum, modelin pozitif sınıfı yakalama başarısının orta seviyede kaldığını göstermektedir.

✓ **Specificity (Özgüllük): 0.50**

Negatif yorumların yarısı doğru şekilde sınıflandırılmıştır. Negatif sınıf için de benzer bir zayıflık gözlemlenmektedir.

✓ **F-Measure (F1-Score): 0.693**

Precision ve recall arasındaki dengeyi gösteren F1 skoru, Naive Bayes'e kıyasla **hafif bir artış** göstermiştir. Ancak bu değer, güçlü bir sınıflandırma performansı için yeterli değildir.

✓ **Cohen's Kappa: 0**

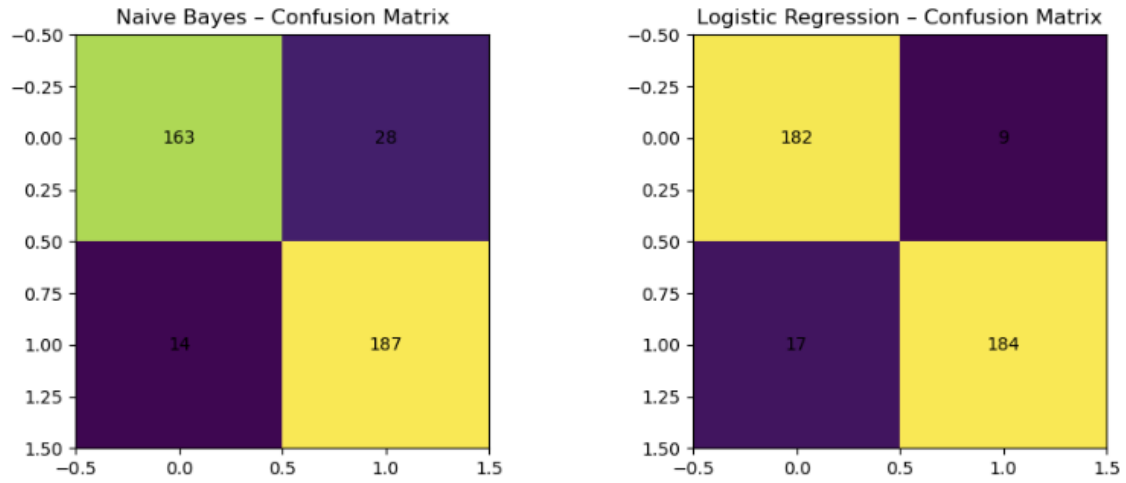
Cohen's Kappa değerinin 0 olması, Logistic Regression modelinin sınıflandırma başarısının **rastgele tahmine anlamlı bir üstünlük sağlamadığını** göstermektedir.

• **Python (Jupyter Notebook) Ortamında Modelleme ve KNIME ile Karşılaştırma**

YouTube Spam veri seti, farklı YouTube kanallarına ait yorum dosyalarının birleştirilmesiyle oluşturulmuştur. Python (Jupyter Notebook) ortamında tüm CSV dosyaları tek bir veri setinde birleştirilmiş ve KNIME tarafında uygulanan veri hazırlama adımlarıyla uyumlu bir yapı elde edilmiştir. Metin verileri TF-IDF yöntemiyle sayısal forma dönüştürülmüş ve sınıflandırma sürecinde Naive Bayes ile Logistic Regression modelleri kullanılmıştır.

Python ortamında yapılan deneylerde, Naive Bayes modeli %89.29 doğruluk oranı elde ederken, Logistic Regression modeli %93.37 doğruluk oranı ile daha başarılı bir performans sergilemiştir. Confusion matrix sonuçları incelendiğinde, Logistic Regression modelinin özellikle yanlış pozitif (false positive) sayısını belirgin şekilde azalttığı görülmektedir. Bu durum, spam olmayan yorumların yanlışlıkla spam olarak etiketlenmesini önlemesi açısından önemlidir.

KNIME ve Python ortamlarında elde edilen sonuçlar birbiriyle tutarlı olup, her iki platformda da Logistic Regression modelinin spam yorumların tespitinde daha uygun bir yaklaşım sunduğu görülmüştür.



Şekil 27 Konfüzyon Matrisleri

Şekil 27’de, YouTube Spam veri seti üzerinde Naive Bayes ve Logistic Regression modellerinin ürettiği konfüzyon matrisleri gösterilmektedir. Naive Bayes modelinde yanlış pozitif sayısının görece yüksek olduğu görülürken, Logistic Regression modelinin yanlış pozitifleri belirgin şekilde azalttığı dikkat çekmektedir. Bu durum, spam olmayan yorumların hatalı şekilde spam olarak sınıflandırılmasını önlemesi açısından Logistic Regression modelinin daha dengeli ve güvenilir bir performans sunduğunu göstermektedir.

Tablo 5’te, YouTube Spam veri seti için KNIME Analytics Platform ve Python (Jupyter Notebook) ortamlarında elde edilen sınıflandırma sonuçları karşılaştırmalı olarak sunulmaktadır. Her iki platformda da TF-IDF temsili kullanılmış ve Naive Bayes ile Logistic Regression modelleri değerlendirilmiştir. Elde edilen sonuçlar, Logistic Regression modelinin her iki ortamda da daha yüksek doğruluk ve daha dengeli hata dağılımı sağladığını ortaya koymaktadır.

Tablo 5 YouTube Spam Veri Seti için KNIME ve Python Sonuçlarının Karşılaştırılması

<i>Kriter</i>	<i>KNIME Analytics Platform</i>	<i>Python (Jupyter Notebook)</i>
<i>Kullanılan Ortam</i>	KNIME	Python (scikit-learn)
<i>Problem Türü</i>	Sınıflandırma	Sınıflandırma
<i>Veri Hazırlama</i>	CSV Birleştirme, Column Filter	CSV Birleştirme (glob), Column Filter
<i>Metin Temsili</i>	TF-IDF	TF-IDF
<i>Kullanılan Modeller</i>	Naive Bayes, Logistic Regression	Naive Bayes, Logistic Regression
<i>Eğitim-Test Bölünmesi</i>	Partitioning	%80 – %20, stratify
<i>Performans Metriği</i>	Accuracy, Confusion Matrix	Accuracy, Confusion Matrix
<i>En İyi Model</i>	Logistic Regression	Logistic Regression
<i>En İyi Accuracy</i>	Yüksek doğruluk	0.9337
<i>Genel Yorum</i>	Yanlış pozitifler düşük	Yanlış pozitifler düşük

- **Sonuç ve Değerlendirme**

Bu çalışmada, YouTube Spam veri seti kullanılarak kullanıcı yorumlarının spam ve spam olmayan olarak sınıflandırılması problemi ele alınmıştır. Analizler hem **KNIME Analytics Platform** hem de **Python (Jupyter Notebook)** ortamında gerçekleştirilmiş ve metin verileri TF-IDF yöntemiyle sayısal forma dönüştürülmüştür. Naive Bayes ve Logistic Regression modelleri kullanılarak yapılan karşılaştırmalar sonucunda, her iki platformda da **Logistic Regression** modelinin daha yüksek doğruluk ve daha dengeli hata dağılımı sunduğu belirlenmiştir.

Confusion matrix sonuçları, Logistic Regression modelinin özellikle yanlış pozitif sayısını azaltarak spam olmayan yorumların hatalı şekilde spam olarak sınıflandırılmasını engellediğini göstermektedir. Bu durum, kullanıcı deneyimi açısından önemli bir avantaj sağlamaktadır. Elde edilen sonuçlar, KNIME ve Python ortamlarında tutarlı olup, metin tabanlı sınıflandırma problemlerinde Logistic Regression modelinin etkin bir çözüm sunduğunu ortaya koymaktadır.